

EMG-BASED PROSTHETIC HAND USING NEURAL NETWORK

A Project Report submitted in partial fulfilment of the requirements

for

the Degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND INSTRUMENTATION ENGINEERING

Submitted

By

Arushi Mallick – 2211100375

Under the guidance of

Dr. Ananya Dastidar

Assistant Professor



SCHOOL OF ELECTRONIC SCIENCES

ODISHA UNIVERSITY OF TECHNOLOGY AND RESEARCH

Session 2025-26

CERTIFICATE

This is to certify that the work contained in the report entitled “**EMG-BASED PROSTHETIC HAND USING NEURAL NETWORK**” is a bonafide work carried out by Arushi Mallick (2211100375) in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Electronics and Instrumentation during the academic session 2025–26.

Supervisor

Head of School
School of Electronic Sciences

DECLARATION

I do hereby declare that the project report entitled “**EMG-BASED PROSTHETIC HAND USING NEURAL NETWORK**”, which has been prepared under the guidance of **Dr Ananya Dastidar, Assistant Professor, School of Electronic Sciences, Odisha University of Technology and Research, Bhubaneswar** is submitted by me for the award of degree of Bachelor of Technology in **Electronics and Instrumentation Engineering**. It is a record of Original and independent research work carried out by me, has not been submitted to any other University/Institution for the award of any Degree or Diploma or published anywhere. References as required have been in the appropriate places.

Date:

Place: Bhubaneswar

Signature

Arushi Mallick (2211100375)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor, **Dr Ananya Dastidar, Assistant Professor, School of Electronic Sciences**, for her continuous guidance, valuable suggestions, and constant encouragement throughout the completion of this project. Her expertise and mentorship were instrumental in shaping our work and ensuring its success.

I also extend our heartfelt thanks to **Dr. Madhab Chandra Tripathy, Head of the Department**, for his motivation, insightful comments, and support during the entire duration of this project.

I express my appreciation to all the **faculty members and staff** of the **School of Electronic Sciences**, Odisha University of Technology and Research, for their cooperation and assistance during the course of this work.

I would like to thank all those authors whose papers and articles have been referred during this project work. Due credit has been given for the figures and data used in this work. Any omission, if any, is inadvertent and not deliberate.

Finally, our gratitude goes to my friends, family, and fellow classmates for their support and cooperation during this period.

Signature

Purna Chandra Swain

Bipul Kumar Sahoo

Arushi Mallick

Aditya Kumar Padhy

ABSTRACT

The loss of a human limb significantly impacts an individual's ability to perform daily activities and reduces overall independence. Prosthetic hands are designed to restore functionality; however, conventional systems often rely on simple control mechanisms and lack intuitive, multi-gesture adaptability. This project presents the design and implementation of an intelligent prosthetic hand system controlled using Electromyography (EMG) signals and a deep learning-based classification model.

The proposed system utilizes five surface EMG sensors to acquire bioelectrical signals generated by forearm muscle activity. These signals, typically in the millivolt range, are captured using differential electrode configuration and conditioned through amplification and bandpass filtering. Further noise reduction is achieved using digital smoothing techniques, followed by segmentation of the signal into overlapping windows for analysis.

From each segment, discriminative time-domain features—including Root Mean Square (RMS), Mean Absolute Value (MAV), Waveform Length (WL), Zero Crossing (ZC), Slope Sign Change (SSC), and Variance (VAR)—are extracted to represent muscle activity patterns. The feature set from multiple channels is normalized and structured into a matrix form, which is then fed into a Convolutional Neural Network (CNN) for multi-class gesture classification.

The CNN model is designed to learn spatial relationships across EMG channels and features, enabling accurate recognition of nine distinct hand gestures, including individual finger movements and combined gestures such as victory sign and thumbs up. The classified output is transmitted via serial communication to an Arduino Nano microcontroller, which actuates five servo motors to replicate the corresponding finger movements using a tendon-driven mechanism.

The integration of biomedical signal processing, embedded systems, and deep learning enables real-time gesture recognition with an achieved classification accuracy of approximately 80%. The system also incorporates confidence-based prediction and sliding window processing to ensure stable and responsive control. Additionally, the architecture is scalable for future integration with IoT-based monitoring systems. This work demonstrates a cost-effective and intelligent approach to prosthetic hand control using EMG and deep learning.

Keywords: *Electromyography (EMG), Prosthetic Hand, Convolutional Neural Network (CNN), Deep Learning, Biomedical Signal Processing, Gesture Recognition, Assistive Technology, Human-Machine Interaction, Internet of Things (IoT)*

CONTENTS

ACKNOWLEDGEMENT.....	4
ABSTRACT.....	5
LIST OF TABLES.....	8
LIST OF FIGURES.....	9
LIST OF ABBREVIATIONS.....	10
Chapter 1: Introduction.....	11
1.1 Background.....	11
1.2 Motivation.....	13
1.3 Objective.....	15
1.4 Report outline.....	16
Chapter 2: Review of EMG-based prosthetic hands.....	17
2.1 Comparative Analysis of Existing Research.....	17
2.2 Research Gap.....	18
2.3 Comparison with Existing Prosthetic Systems.....	19
2.4 Summary.....	19
Chapter 3: Methodology.....	21
3.1 System Overview.....	21
3.2 Hardware Design.....	22
3.3 Bio signal Acquisition and Analog Signal Conditioning.....	27
3.4 Software Requirements.....	31
3.5 Signal Processing and Feature Extraction.....	32
3.6 Convolutional Neural Network (CNN) Architecture.....	35
3.7 Live Prediction and Finger Movement Control.....	38
3.8 Data Collection for Generic EMG Database.....	41
3.9 Summary.....	41
Chapter 4: Implementation of a prosthetic arm.....	43
4.1 EMG Signal Observation.....	43
4.2 Results and Observations.....	45
4.3 System Performance.....	48
4.4 Summary.....	49
Chapter 5: Conclusions and Future Works.....	50
5.1 Conclusion.....	50

5.2	Future Work.....	50
	References.....	52
	Appendix A: Source Code.....	56

LIST OF TABLES

Table 2.1 Comparison of Existing Research Works.....	17
Table 2.2 Comparison with Existing Prosthetic Systems.....	19
Table 3.1 List of Components and their function.....	22
Table 4.1 Observed ADC value ranges for different finger movements across five EMG channels.....	44
Table 4.2 Generic Model Performance	45
Table 4.3 Custom Model Performance.....	47
Table 4.4 Performance Metrics Comparison.....	48

LIST OF FIGURES

Fig. 1.1 Block diagram of the proposed EMG-based prosthetic hand system.....	12
Fig. 3.1 Block diagram of the proposed EMG-based prosthetic hand system.....	21
Fig. 3.2 EMG Sensor: Muscles BioAmp Patchy.....	23
Fig. 3.3 Arduino Nano IC pin out	24
Fig. 3.4 Servo Motor MG996R and it's dimension	25
Fig. 3.5 3D-printed components of the prosthetic hand and forearm	25
Fig. 3.6 3D-printed prosthetic hand.....	26
Fig. 3.7 Circuit Connection.....	26
Fig. 3.8 Diagram for EMG sensor placement.....	28
Fig. 3.9 Datasheet of Muscle BioAmp Patchy.....	30
Fig. 3.10 EMG sensor connected to Arduino Nano	30
Fig. 3.11 Convolutional Neural Network architecture for EMG-based gesture classification	35
Fig. 3.12 Flowchart of the EMG-based prosthetic hand control system	39
Fig. 4.1 EMG signal patterns (ADC values) for different finger movements across five channels.	44
Fig. 4.2 Data Collection and Testing of Model	46
Fig. 4.3 Confusion Matrix and ROC Curve	48

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
CAD	Computer-Aided Design
CNN	Convolutional Neural Network
CMRR	Common Mode Rejection Ratio
DC	Direct Current
DOF	Degree of Freedom
EMG	Electromyography
IoT	Internet of Things
InAmp	Instrumentation Amplifier
IDE	Integrated Development Environment
MAV	Mean Absolute Value
MCU	Microcontroller Unit
ML	Machine Learning
PLA	Polylactic Acid
PWM	Pulse Width Modulation
REF	Reference Electrode
ReLU	Rectified Linear Unit
RMS	Root Mean Square
SNR	Signal-to-Noise Ratio
SSC	Slope Sign Change
USB	Universal Serial Bus
VAR	Variance
VCC	Voltage Common Collector (Power Supply Voltage)
WL	Waveform Length
ZC	Zero Crossing

Chapter 1: Introduction

1.1 Background

Advancements in biomedical engineering, robotics, and artificial intelligence have significantly expanded the possibilities for developing intelligent and affordable prosthetic devices. Prosthetic hands play a vital role in restoring lost limb functionality, enabling amputees to regain independence and perform daily activities. However, commercially available myoelectric prosthetic limbs are often prohibitively expensive, typically ranging from ₹5 to ₹15 lakhs, making them inaccessible to a large population, particularly in developing countries. This economic barrier has driven research toward low-cost, customizable, and technologically advanced alternatives .

Electromyography (EMG) is a widely used biomedical signal acquisition technique for capturing the electrical activity generated by skeletal muscles. When muscle fibers contract, they produce electrical potentials typically in the range of 0.5 mV to 5 mV. These signals can be detected non-invasively from the skin surface using surface electrodes. EMG signals are inherently stochastic, non-stationary, and highly susceptible to noise, making their processing and interpretation a challenging task in real-time control applications.

In this project, EMG signals are acquired using the Muscles BioAmp Patchy sensor, a wearable biopotential acquisition module designed for surface EMG recording. The sensor operates based on a multi-stage signal conditioning pipeline. Initially, differential electrodes (IN+ and IN-) capture the muscle activity, while a reference electrode (REF) provides a stable ground. The captured signal is then passed through an instrumentation amplifier, which amplifies the differential signal while rejecting common-mode noise such as power line interference. Following amplification, the signal undergoes analog filtering using an active bandpass filter. The filter is designed with a high-pass cutoff around 74.5 Hz to eliminate motion artifacts and DC drift, and a low-pass cutoff around 149.5 Hz to suppress high-frequency noise. This ensures that the signal is confined within the effective EMG frequency band, thereby improving signal quality for further processing. Despite hardware-level filtering, additional digital preprocessing is required. In this system, noise reduction is further achieved through averaging and moving average filtering techniques implemented in software. The EMG signals are then segmented into overlapping windows to capture temporal variations in muscle activity. From each

segment, a set of discriminative time-domain features is extracted, including Root Mean Square (RMS), Mean Absolute Value (MAV), Waveform Length (WL), Zero Crossing (ZC), Slope Sign Change (SSC), and Variance (VAR). These features provide a compact yet informative representation of the EMG signal characteristics.

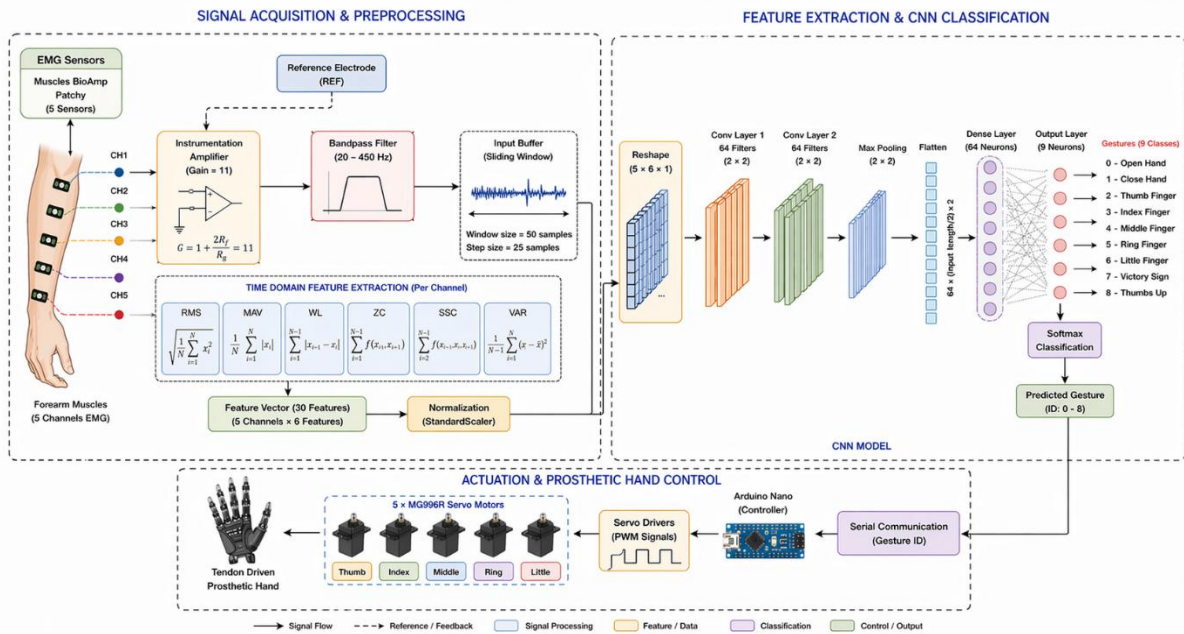


Fig. 1.1 Block diagram of the proposed EMG-based prosthetic hand system

To enable intelligent gesture recognition, a Convolutional Neural Network (CNN) is employed as the classification model. Unlike traditional machine learning approaches, CNNs can learn spatial relationships between features and extract hierarchical representations. In this work, the extracted features from five EMG channels are arranged into a structured input matrix and fed into a CNN model consisting of multiple convolutional, batch normalization, pooling, and fully connected layers. The network performs multi-class classification to identify nine distinct hand gestures, including individual finger movements and combined gestures such as victory sign and thumbs up.

The classified output is transmitted via serial communication to an Arduino Nano microcontroller, which controls five MG996R servo motors. Each motor corresponds to a specific finger, enabling precise actuation based on the predicted gesture. The mechanical structure of the prosthetic hand is fabricated using 3D printing, incorporating a tendon-driven mechanism that mimics natural finger flexion and extension. The integration of biomedical signal processing, embedded systems, and deep learning enables real-time gesture recognition

with an achieved accuracy of approximately 80%. The system demonstrates the capability of translating complex muscle activity patterns into meaningful control signals for prosthetic actuation. Furthermore, the architecture is designed with scalability in mind, allowing future integration with Internet of Things (IoT) platforms for remote monitoring, adaptive learning, and performance analysis.

This project highlights the potential of combining EMG-based human-machine interfaces with artificial intelligence to develop intelligent, cost-effective, and user-friendly assistive devices. In addition to system development, this work emphasizes the importance of creating a generalized EMG dataset by collecting signals from multiple individuals with varying physical characteristics. Unlike conventional approaches that rely on user-specific training, a multi-user dataset enables the model to learn common patterns of muscle activity rather than individual variations. This enhances the robustness and generalization capability of the system, allowing it to operate with minimal or no retraining for new users. Such an approach is particularly beneficial for practical prosthetic applications, where ease of use and immediate deployment are critical.

Overall, this work presents a comprehensive approach toward next-generation prosthetic systems by bridging the gap between biomedical signal acquisition, intelligent computation, and physical actuation.

1.2 Motivation

The development of the EMG-based prosthetic hand using a neural network is motivated by a combination of social, technological, and practical challenges associated with existing prosthetic systems. The key motivating factors are discussed below:

1. High Cost of Commercial Prosthetic Devices

Advanced myoelectric prosthetic hands available in the market are highly sophisticated but extremely expensive, often costing between ₹5 to ₹15 lakhs. Such devices are unaffordable for a large section of the population, particularly in developing countries. This creates a significant gap between technological advancements and real-world accessibility, motivating the need for low-cost alternatives.

2. Limitations of Conventional Control Systems

Traditional low-cost prosthetic hands typically use simple threshold-based control of EMG signals, allowing only basic operations such as opening and closing of the hand. These systems lack flexibility and cannot support multiple independent finger movements or complex gestures. This limitation reduces their usability in real-life scenarios where diverse hand functions are required.

3. Need for Intelligent and Adaptive Control

Human hand movements are highly complex and require precise coordination. To replicate such functionality, prosthetic systems must be capable of interpreting subtle variations in muscle signals. The use of deep learning techniques, such as Convolutional Neural Networks (CNN), enables accurate classification of multiple gestures from EMG signals, providing more natural and intuitive control compared to conventional approaches.

4. Integration of Biomedical Signals with Artificial Intelligence

The combination of EMG signal processing and machine learning represents a significant advancement in assistive technology. Such integration enhances system performance and opens new possibilities for human-machine interaction.

5. Advancement in Low-Cost Hardware and Open-Source Platforms

The availability of affordable components such as Arduino Nano, EMG sensors (Muscles BioAmp Patchy), and high-torque servo motors enables the development of cost-effective prosthetic devices. Additionally, open-source tools like Arduino IDE and VS Code simplify development, encourage experimentation, and support rapid prototyping.

6. Functional Rehabilitation and Quality of Life Improvement

A prosthetic hand capable of performing multiple gestures significantly improves the daily life of amputees. It allows users to perform essential tasks such as gripping, pointing, and gesturing, thereby enhancing independence, confidence, and social interaction.

7. Scalability and Future Integration with IoT

Modern assistive devices are increasingly connected and data-driven. This project is designed with scalability in mind, allowing future integration with IoT platforms for remote monitoring, performance tracking, and adaptive learning. This adds an additional layer of intelligence and usability to the system.

So, the motivation behind this project lies in bridging the gap between cost and functionality, while leveraging modern technologies such as EMG signal processing and neural networks to create an intelligent, responsive, and user-friendly prosthetic hand.

1.3 Objective

The primary objective of this project is to design and develop an intelligent, multi-channel EMG-based prosthetic hand system capable of accurately classifying multiple hand gestures using a Convolutional Neural Network (CNN) and executing corresponding movements through real-time actuation.

To achieve this, the following detailed technical objectives are defined:

1. Design and Fabrication of Prosthetic Hand Mechanism

- To develop a 3D-printed prosthetic hand structure based on a modular and lightweight design.
- To implement a tendon-driven actuation mechanism using flexible transmission (e.g., fishing lines) for realistic finger movement.
- To ensure mechanical compatibility with servo motor torque and range of motion.

2. Multi-Channel EMG Signal Acquisition

- To acquire bioelectrical signals from forearm muscles using five surface EMG sensors (Muscles BioAmp Patchy).
- To utilize differential electrode configuration (IN+, IN−, REF) for accurate signal acquisition.
- To interface analog EMG outputs with the Arduino Nano for real-time sampling.

3. Analog Signal Conditioning and Noise Suppression

- To utilize the built-in instrumentation amplifier for differential signal amplification and common-mode noise rejection.
- To apply hardware-level active bandpass filtering (approximately 74.5 Hz to 149.5 Hz) to isolate the EMG frequency band.
- To minimize motion artifacts, baseline drift, and power line interference during acquisition.

4. Digital Signal Preprocessing

- To implement software-based noise reduction techniques, including signal averaging and moving average filtering.
- To segment continuous EMG signals into overlapping windows (window size = 50 samples, step size = 25 samples) for temporal analysis.
- To ensure stable and consistent signal representation for feature extraction.

6. Model Training and Performance Evaluation

- To train the CNN model using supervised learning with appropriate loss functions and optimizers.
- To evaluate model performance using metrics such as accuracy, confusion matrix, and ROC curves.

7. Real-Time Gesture Prediction System

- To implement a real-time inference pipeline for continuous EMG signal processing and classification.
- To minimize latency for smooth and responsive prosthetic control.

8. Development of Generic EMG Dataset

- To collect EMG data from multiple individuals with varying physical characteristics to build a generalized dataset.
- To reduce dependency on user-specific training and enable immediate usability for new users.
- To improve model robustness and adaptability across different users and muscle conditions.

9. Embedded Control System Integration

- To transmit predicted gesture commands to the Arduino Nano via serial communication.
- To map gesture classes to corresponding servo motor control signals.

10. Multi-Gesture Prosthetic Control Implementation

- To enable the prosthetic hand to perform nine functional gestures: open hand, individual finger movements, closed hand, victory sign, and thumbs up.
- To ensure precise, stable, and repeatable finger actuation.

So, the objective of this project is to develop a cost-effective, intelligent, and scalable prosthetic hand system that leverages EMG signal processing and deep learning to achieve accurate and natural human-machine interaction.

1.4 Report outline

Chapter 1 details the introduction while Chapter 2 discusses the various literature and their limitations. Chapter 3 provide the methodology and Chapter 4 the implementation and results. Conclusions and the future scopes have been presented in Chapter 5.

Chapter 2: Review of EMG-based prosthetic hands

Electromyography (EMG)–based prosthetic hands represent a major advance in restoring functional ability to individuals with upper-limb loss. These systems rely on bioelectrical signals generated by residual muscles, which are detected through surface or implanted electrodes. The EMG signals are processed and translated into control commands for actuating prosthetic fingers and joints. A review of different literature has been carried out in this chapter.

2.1 Comparative Analysis of Existing Research

A number of research works have been carried out in the field of EMG-based prosthetic hands, focusing on signal acquisition, feature extraction, and classification techniques. A comparative analysis of selected research papers is presented in Table 2.1.

Table 2.1 Comparison of Existing Research Works

Sl. No.	Author(s)/ Year	Study/Paper	Methodology/Device	Results, Performance and Limitations
1	Korai et al., 2022	Using 3D Printing Technology to Develop Electromyography Signals Based Prosthetic Arm.	3D-printed PLA arm, individual servo per finger (MG995), Arduino UNO, MyoWare-style EMG input; CAD + PLA printing, fishing-line tendon actuation.	Demonstrated a low-cost (~ \$100) EMG-controlled arm with $\approx 92\%$ open/close accuracy; good learning tool. Limited to single-channel EMG, basic threshold control, and moderate grip strength due to PLA
2	Ashik Ali et al., 2020	Design and Development of EMG Based Prosthetic Arm	3-channel EMG acquisition, instrumentation amplifier + filters, Arduino-based controller; servos for actuation; 3-channel data used for multi-gesture recognition; prototype implementation described.	Shown improved gesture recognition using multi-channel EMG; highlights need for filtering & feature extraction. Limited by noise, slow real-time response, and basic microcontroller capability.
3	Unanyan & Belov, 2021	Low-Price Prosthetic Hand Controlled by EMG Signals.	3D printed mechanical design , Arduino Nano electronics, DHV56MG servos (reducer used), multi-finger control and pattern-recognition software; emphasis on mechanical strength and grip force comparable to healthy hand.	Produced strong, near-natural grip force with modular structure. Slightly heavier and mechanically complex, limited testing on real users.
4	Khan et al., 2021	Supervised Machine Learning based Fast Hand Gesture Recognition and Classification Using EMG Signal.	Multi-channel EMG (Myo armband), spectral features, EMG preprocessing, Cubic-SVM classifier	98.9% Accuracy. Limited gestures (4), mostly offline/simulation, limited real-time prosthetic integration

From Table 2.1 we see that early designs offered limited open–close functionality, but modern systems support multiple grip patterns and finer dexterity. Signal acquisition and processing remain challenging due to noise, muscle fatigue, and electrode displacement. Advanced algorithms, including pattern recognition and machine learning, have significantly improved control accuracy and responsiveness. Integration with microcontrollers enables real-time interpretation and actuation, enhancing usability in daily tasks. Researchers are also exploring sensory feedback mechanisms to provide users with a sense of touch and force. Despite progress, issues such as cost, training complexity, and long-term reliability persist. Overall, EMG-based prosthetic hands continue to evolve toward more intuitive, adaptive, and human-like control systems.

2.2 Research Gap

Despite significant advancements in EMG-based prosthetic systems, several limitations remain in existing research:

1. Limited Number of Gestures

Many systems support only basic actions such as open/close or a small number of gestures (2–6), restricting functionality.

2. Lack of Real-Time Implementation

High-accuracy models (e.g., SVM, CNN) are often tested in simulation environments and not integrated with real-time prosthetic hardware.

3. High Cost of Advanced Systems

Commercial prosthetic devices with advanced control capabilities are expensive and not accessible to users in developing regions.

4. Simplistic Signal Processing

Several studies rely on basic feature extraction and threshold-based control, limiting classification performance.

5. Limited Adaptability and Intelligence

Many systems lack adaptive learning and cannot generalize across different users or varying muscle conditions.

Contribution of Proposed System

The proposed system addresses these gaps by:

- Supporting 9 distinct hand gestures
- Implementing real-time CNN-based classification
- Using multi-channel EMG (5 sensors)
- Providing a low-cost and scalable solution

2.3 Comparison with Existing Prosthetic Systems

A comparison between existing prosthetic solutions (including commercial systems and SVNIRTAR Olatpur) and the proposed system is presented in Table 2.2.

Table 2.2 Comparison with Existing Prosthetic Systems

Parameter	SVNIRTAR (Olatpur)	Symbionic	Commercial Robotic Arm	Proposed System
Cost	~₹10,000	~₹2,00,000	~₹8,00,000	~₹20,000
Control Type	Mechanical / basic EMG	Myoelectric	Advanced robotic	CNN-based EMG
Intelligence	No AI	Limited	Moderate	AI-based (Deep Learning)
Number of Gestures	1–2	Multiple grips	Multiple	9 gestures
Adaptability	Low	Moderate	Moderate	High
Real-Time Control	Yes	Yes	Yes	Yes
Customization	Limited	Moderate	Low	High

Prosthetic devices provided at SVNIRTAR, Olatpur are highly affordable and accessible, making them suitable for rehabilitation purposes. However, they typically offer limited movement and lack intelligent control mechanisms. Commercial systems such as Symbionic and high-end robotic arms provide improved functionality but are significantly expensive, limiting their accessibility. The proposed system provides a balanced solution by combining Affordability, Multiple gesture control, Artificial intelligence-based decision making

2.4 Summary

This section presented a comparative analysis of existing EMG-based prosthetic hand systems, highlighting the methodologies, performance, and limitations of prior research. It was observed that many existing systems either support a limited number of gestures, rely on basic signal processing techniques, or lack real-time hardware implementation. While advanced machine learning models have demonstrated high accuracy, they are often restricted to simulation environments and are not fully integrated into practical prosthetic systems.

The analysis also revealed a significant trade-off between cost and functionality in currently available prosthetic devices. Low-cost solutions, such as those provided by SVNIRTAR

(Olatpur), offer limited control and lack intelligent features, whereas commercial systems provide better functionality but are expensive and less accessible.

The identified research gaps emphasize the need for a system that is affordable, intelligent, and capable of real-time multi-gesture control. The proposed system addresses these challenges by integrating multi-channel EMG acquisition, CNN-based classification, and embedded actuation, achieving a balance between cost, performance, and usability. This makes it a practical and scalable solution for advanced prosthetic hand development.

Chapter 3: Methodology

This chapter describes the methodology used in the design and development of an electromyography (EMG)-based prosthetic arm that enables hand motion through muscle activity signals. The primary objective of this work is to develop a low-cost, 3D-printed, and Arduino-based prosthetic arm capable of performing basic open and close operations in response to muscle contractions detected from the forearm using a single EMG sensor. The chapter details the system architecture, hardware components, circuit interfacing, software development, and testing procedures followed during implementation.

3.1 System Overview

The proposed system is designed as an end-to-end pipeline that converts human muscle activity into controlled prosthetic hand movements using EMG signal processing and deep learning. The system consists of four major modules: signal acquisition, signal processing, gesture classification, and actuation.

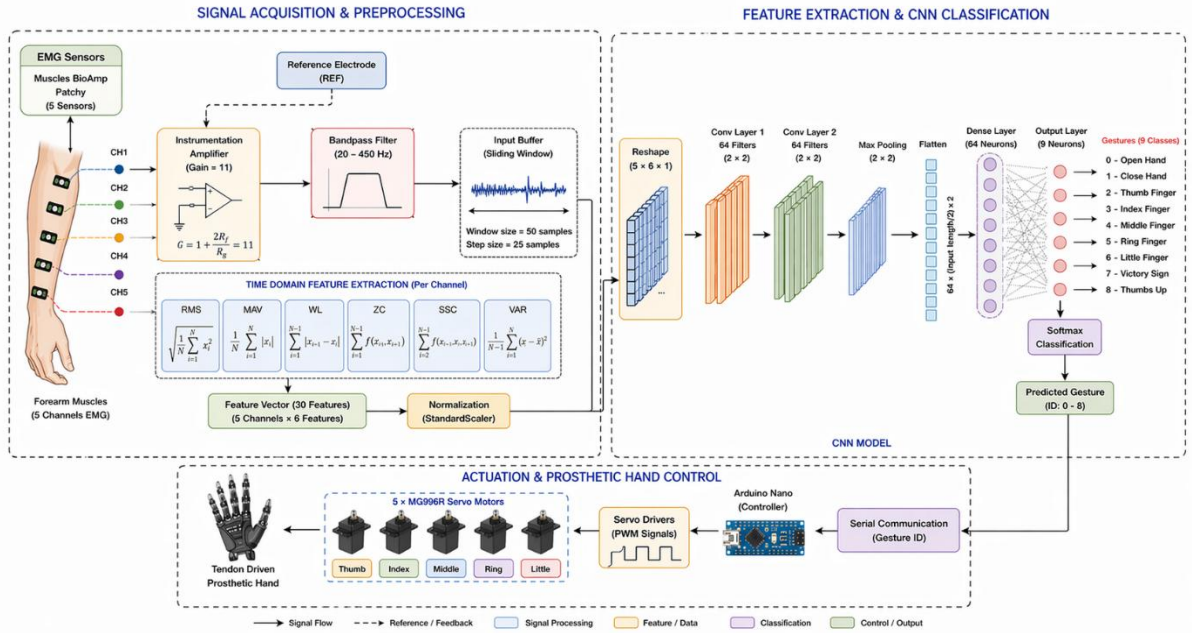


Fig. 3.1 Block diagram of the proposed EMG-based prosthetic hand system

Initially, EMG signals are acquired from the user's forearm using five BioAmp Patchy sensors. These sensors capture the electrical activity generated during muscle contractions associated with different finger movements. The signals are conditioned through amplification and bandpass filtering to improve signal quality.

The conditioned signals are transmitted to a processing unit, where digital preprocessing and feature extraction are performed. The extracted features are normalized and structured into a matrix form, which serves as input to the CNN model.

The CNN processes the input and classifies it into one of nine predefined gestures. This classification represents the intended movement of the user. The predicted gesture is then transmitted to the Arduino Nano via serial communication.

Upon receiving the command, the Arduino controls five servo motors that actuate the prosthetic fingers. Each motor is mapped to a specific finger, enabling both individual and combined finger movements.

The system operates in real-time using a sliding window mechanism, ensuring continuous and smooth gesture recognition. Additionally, a confidence-based filtering mechanism is used to improve reliability and prevent unintended activations.

Overall, the system provides a seamless interface between human muscle activity and mechanical actuation, demonstrating an effective integration of biomedical signal processing, machine learning, and embedded systems.

3.2 Hardware Design

3.2.1 Main Components

The key components required for EMG-based prosthetic hands include surface or implanted electrodes for signal acquisition, amplification and filtering circuits, a microcontroller or processor for signal processing, and actuators to drive the prosthetic fingers. Additionally, power supply units, sensors for feedback, and communication interfaces are used to ensure efficient control and functionality of the system.

Table 3.1: List of Components and their function

Component	Specification / Function
Arduino Nano	ATmega328P microcontroller, 16 MHz clock, 10-bit ADC; used for EMG data acquisition and servo control via serial commands
EMG Sensors (Muscles BioAmp Patchy × 5)	Surface EMG sensors for detecting muscle activity; built-in amplification and bandpass filtering; analog output

Servo Motors (MG996R × 5)	High-torque metal gear servos; used for individual finger actuation
Power Supply	2×3.7 V Li-ion batteries (7.4 V) for servos; Arduino powered via USB or regulated 5 V
EMG Surface Electrodes	Disposable Ag/AgCl ECG/EMG Electrodes
3D-Printed Prosthetic Hand	PLA-based mechanical structure with tendon-driven finger mechanism
Connecting Components	Jumper wires, electrodes, fishing line, screws, connectors for assembly and wiring

EMG Muscle Sensor

The Muscles BioAmp Patchy is a compact, wearable surface electromyography (sEMG) sensor designed for direct skin contact using gel electrodes. It enables real-time acquisition of muscle activity signals with integrated analog signal conditioning.

The sensor operates using a **three-electrode differential configuration**:

- IN+ and IN− electrodes capture differential muscle activity
- REF electrode provides a stable reference ground



Fig. 3.2 EMG Sensor: Muscles BioAmp Patchy [11]

Internally, the sensor consists of:

- **Instrumentation Amplifier:** Amplifies low-amplitude EMG signals (in millivolt range) while rejecting common-mode noise such as power line interference.
- **Active Bandpass Filter:** Restricts the signal to a specific frequency range (~74.5 Hz to ~149.5 Hz), removing motion artifacts and high-frequency noise.
- **Signal Conditioning Circuit:** Produces a stable analog output suitable for microcontroller input.

The output of the sensor is an analog voltage proportional to muscle contraction intensity, which is fed into the analog input pins of the Arduino Nano.

Arduino Nano

The Arduino Nano is a compact microcontroller board based on the ATmega328P. It operates at 16 MHz and provides:

- 8 analog input pins with 10-bit ADC resolution
- Multiple digital I/O pins with PWM capability
- UART-based serial communication

In this system, the Arduino Nano performs the following functions:

- Reads analog EMG signals from five sensors
- Applies basic signal averaging to reduce noise
- Transmits digitized EMG data to a processing unit (computer) via serial communication
- Receives classified gesture commands from the CNN model
- Generates PWM signals to control servo motors

The Nano is preferred due to its compact size, sufficient I/O capability, and ease of integration.

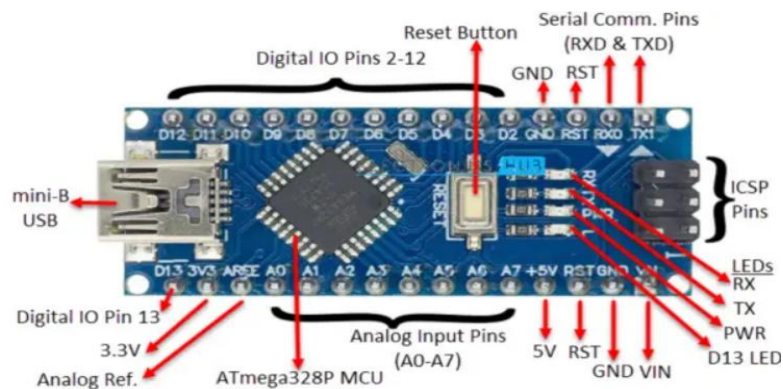


Fig. 3.3 Arduino Nano IC pin out [13]

Servo Motor (MG996R)

The MG996R is a high-torque, metal-gear servo motor commonly used in robotics and prosthetic applications for precise angular movement. Here, it is mainly used for moving finger of the prosthetic hand system. It operates on a 4.8–7.2 V supply and is controlled through a Pulse Width Modulation (PWM) signal from a microcontroller such as the Arduino. Key internal components include:

- **DC Motor with Metal Gear Train:** Provides high torque (up to ~10–12 kg·cm) and durability
- **Feedback Potentiometer:** Ensures accurate position control through closed-loop feedback
- **Control Circuit:** Interprets PWM signals (1–2 ms pulse width) to determine angular position.

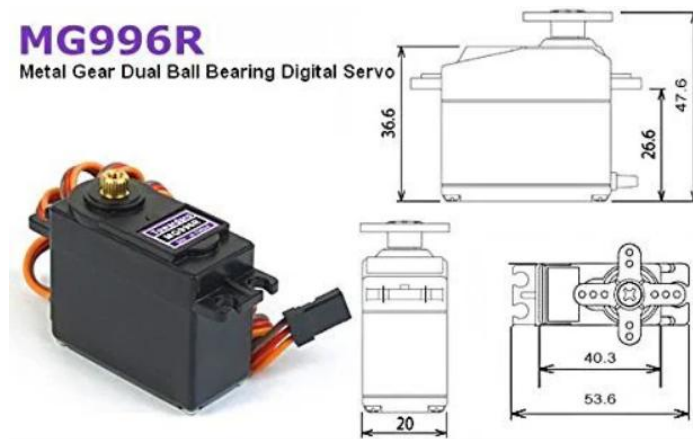


Fig. 3.4 Servo Motor MG996R and it's dimension [15]

The MG996R provides up to 10–12 kg·cm torque at 6 V, a rotation range of approximately 0°–180°, and fast response, making it well-suited for finger actuation in the 3D-printed prosthetic hand.

3D-Printed Hand

The 3D-printed prosthetic hand and forearm were fabricated using an open-source InMoov design, chosen for its modularity and human-like motion. All components were printed using Polylactic Acid (PLA) filament.

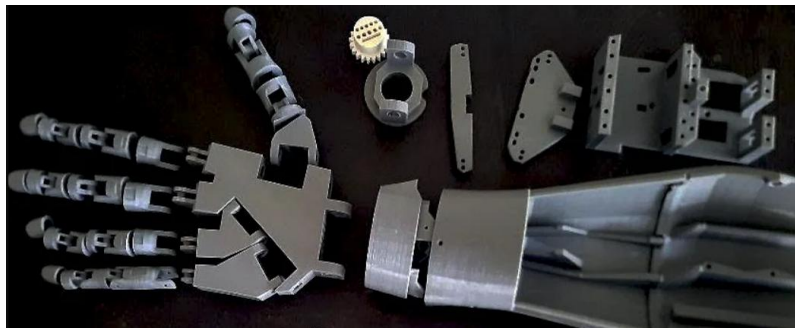


Fig. 3.5 3D-printed components of the prosthetic hand and forearm

Each finger consists of multiple articulated segments connected by joints and actuated through nylon fishing-line tendons linked to servo motors housed inside the forearm. The palm and forearm were designed to securely hold five servos while maintaining ergonomic balance and reduced overall weight. 3D printing offers the advantages of low cost, rapid prototyping, easy customization, and lightweight construction, making it an ideal method for prosthetic design and experimentation. The printed assembly enables smooth opening and closing of all fingers in response to EMG-based control signals.

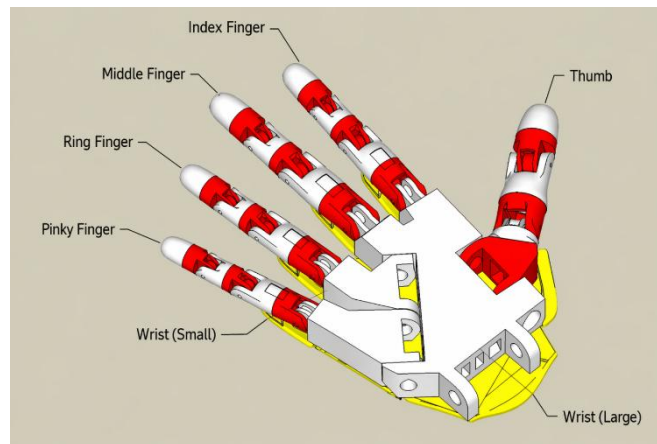


Fig. 3.6 3D-printed prosthetic hand

3.2.2 Circuit Connections

1. EMG Sensors

- Sensor outputs (SIG) → Arduino analog pins (A0 to A4)
- VCC → 3.3 V or 5 V (as per sensor specification)
- GND → Common ground

2. Servo Motors

- Signal pins → Digital PWM pins (D3, D5, D6, D9, D10)
- VCC → External 7.4 V battery positive terminal
- GND → Shared common ground with Arduino

3. Arduino Nano Power

- Powered via USB connection or regulated 5 V input
- Ensures stable logic-level operation

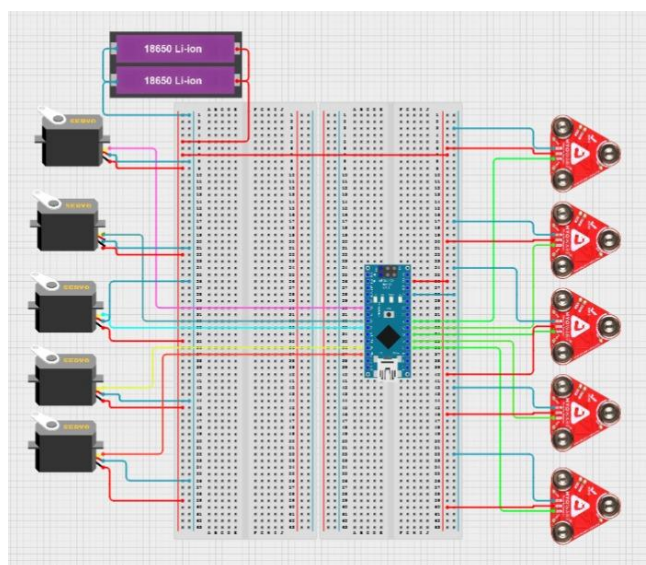


Fig. 3.7 Circuit Connection

3.3 Bio signal Acquisition and Analog Signal Conditioning

Electromyography (EMG) is a biomedical signal acquisition technique used to measure the electrical activity generated by skeletal muscles during contraction. When motor neurons activate muscle fibers, ion exchange across muscle membranes produces electrical potentials, typically ranging from 0.5 mV to 5 mV. These signals are non-stationary, stochastic in nature, and highly susceptible to noise and interference, making accurate acquisition and conditioning essential for reliable prosthetic control.

3.3.1 Sensor Placement and Muscle Identification

Proper placement of EMG sensors is essential for accurate signal acquisition and reliable gesture recognition. In the proposed system, sensors are positioned on specific forearm muscles responsible for controlling individual finger movements. The forearm consists of multiple muscle groups involved in flexion and extension, and each sensor is strategically placed to capture activity from targeted muscle regions.

The placement is performed based on the following considerations:

- Sensors are aligned along the direction of muscles to maximize signal strength
- Adequate spacing is maintained between sensors to minimize cross-talk
- The reference electrode (REF) is placed on a bony or electrically neutral region to ensure stable grounding
- Skin preparation (cleaning and proper electrode attachment) is performed to reduce impedance and improve signal quality

Based on anatomical muscle distribution, sensors are positioned to capture signals corresponding to:

- Thumb movement (thenar and extensor muscles)
- Index finger movement (flexor digitorum muscles)
- Middle finger movement
- Ring finger movement
- Little finger movement (hypothenar region)

Accurate muscle identification and consistent sensor placement significantly improve signal reliability, repeatability, and classification performance.

3.3.2 EMG Signal Acquisition

In the proposed system, EMG signals are acquired using five Muscles BioAmp Patchy sensors. These are wearable surface EMG sensors designed for direct attachment to gel electrodes, enabling compact and stable signal acquisition from the forearm muscles.

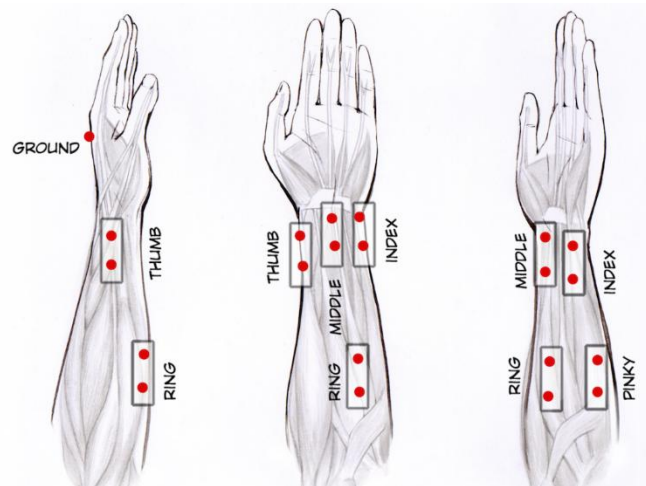


Fig. 3.8 Diagram for EMG sensor placement [12]

Each sensor operates using a **three-electrode configuration**:

- **IN+ (non-inverting electrode):** Placed over the active muscle region
- **IN- (inverting electrode):** Placed along the same muscle fiber direction
- **REF (reference electrode):** Placed on a bony or electrically neutral region (e.g., elbow)

The differential voltage between IN+ and IN- represents the EMG signal, while the REF electrode establishes a stable baseline reference for the body. Proper electrode placement is critical, as it directly affects signal quality, selectivity, and repeatability.

3.3.3 Signal Characteristics and Challenges

EMG signals exhibit the following characteristics:

- Low amplitude (microvolt to millivolt range)
- Frequency content typically between 20 Hz and 500 Hz
- Susceptibility to noise sources such as:
 - Power line interference (50/60 Hz)
 - Motion artifacts
 - Electrode-skin impedance variations
 - Cross-talk from adjacent muscles

Due to these challenges, raw EMG signals cannot be used directly and must undergo analog signal conditioning.

3.3.4 Instrumentation Amplifier (Differential Amplification)

The first stage of signal conditioning in the BioAmp Patchy sensor is an **instrumentation amplifier (InAmp)**. This stage performs differential amplification of the EMG signal:

- Amplifies only the difference between $IN+$ and $IN-$
- Rejects common-mode signals such as environmental noise

The effectiveness of this stage is characterized by a high **Common Mode Rejection Ratio (CMRR)**, which is crucial for eliminating interference such as power line noise. Since EMG signals are extremely small, this amplification stage significantly improves the signal-to-noise ratio (SNR) before further processing.

3.3.5 Active Bandpass Filtering

Following amplification, the signal passes through an **active bandpass filter** implemented using operational amplifiers. The filter is designed to isolate the relevant EMG frequency band while suppressing unwanted components.

The bandpass filter consists of:

(a) High-Pass Filter (~74.5 Hz)

- Removes low-frequency components such as:
 - Motion artifacts
 - Baseline drift
 - DC offsets
- Ensures that slow variations do not distort the signal

(b) Low-Pass Filter (~149.5 Hz)

- Attenuates high-frequency noise including:
 - Electromagnetic interference
 - High-frequency electronic noise
- Preserves the dominant EMG frequency components

Together, these filters create an effective passband (~74.5 Hz to ~149.5 Hz), which captures the most informative portion of the EMG signal for gesture recognition.

3.3.6 Role of Analog Signal Conditioning

The analog signal conditioning stage plays a critical role in ensuring reliable EMG acquisition:

- Enhances signal amplitude to a usable range
- Improves signal-to-noise ratio (SNR)
- Removes unwanted frequency components before digitization
- Reduces computational burden on digital processing stages

The figure is divided into four quadrants by dashed lines, each containing a circuit diagram or component information.

- Top-Left Quadrant:** A circuit diagram for an **Instrumentation amplifier (Gain: x11)**. It features a central op-amp with two input buffers and a feedback network. Components include resistors of 10k, 1k, and 100k, and capacitors of 10nF and 100nF.
- Top-Right Quadrant:** A circuit diagram for a **BandPass filter (Gain: x220)**. It shows a multi-stage filter with op-amps, resistors (10k, 1k, 100k), and capacitors (10nF, 100nF).
- Bottom-Left Quadrant:** A circuit diagram for a **Power supply**. It shows a voltage divider with resistors of 10k, 1k, and 100k, and capacitors of 10nF and 100nF.
- Bottom-Right Quadrant:** A circuit diagram for **Connectors**. It shows a connector with pins 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100.

Table:

File	Music-Bioamp-Patchy.kicad.sch	Rev	1.0
Sheet	1	1	1
File	Music-Bioamp-Patchy.kicad.sch	Rev	1.0
Sheet	1	1	1

3.3.7 Interface with Microcontroller

An illustration of a person's arm with a small red sensor module attached to the elbow. The sensor is connected via four colored wires (yellow, red, black, and white) to a blue microcontroller board. The board features a USB port, a black diamond-shaped component, and several pins.

Additionally, a simple averaging technique is applied at the microcontroller level to reduce high-frequency fluctuations and improve signal stability before transmission to the processing unit.

3.4 Software Requirements

The development and implementation of the proposed EMG-based prosthetic hand system require a combination of software tools for signal acquisition, processing, machine learning, and embedded system control. The following software and libraries were used:

3.4.1 Development Environments

1. Visual Studio Code (VS Code):

Used as the primary development environment for writing and executing Python code. It provides support for extensions, debugging tools, and integrated terminal functionality, making it suitable for machine learning and data processing tasks.

2. Arduino IDE:

Used for programming the Arduino Nano microcontroller. It enables code compilation, uploading, and serial communication for real-time EMG data acquisition and servo motor control.

Programming Language

Python:

Used for data processing, feature extraction, model training, and real-time prediction. Python was chosen due to its extensive support for scientific computing and machine learning.

3.4.2 Libraries and Frameworks

1. NumPy:

Used for numerical computations and handling EMG signal arrays.

2. Pandas:

Used for dataset handling, preprocessing, and structured data manipulation.

3. Matplotlib / Pyplot:

Used for visualization of EMG signals, training curves, and performance graphs.

4. Scikit-learn:

Used for feature scaling (StandardScaler), model evaluation metrics, and dataset splitting.

5. TensorFlow:

Used to design, train, and deploy the Convolutional Neural Network (CNN) model for gesture classification.

6. Joblib:

Used for saving and loading trained models and preprocessing objects such as scalers.

7. **PySerial:**

Used for serial communication between the computer and Arduino Nano to receive real-time EMG data.

Other Tools

- **Serial Plotter (Arduino IDE):**

Used for real-time visualization of EMG signals during data acquisition and testing.

3.5 Signal Processing and Feature Extraction

The acquired EMG signals are inherently noisy, non-stationary, and sensitive to external interference. Therefore, an effective digital signal processing pipeline is essential to extract meaningful information for accurate gesture classification. The proposed system implements a multi-stage processing approach consisting of preprocessing, segmentation, feature extraction, and normalization.

3.5.1 Digital Signal Preprocessing

After analog signal conditioning, the EMG signals are digitized by the Arduino Nano and transmitted to a computer via serial communication. Digital preprocessing is applied to further enhance signal quality and prepare the data for feature extraction.

(a) Averaging (Arduino)

To reduce high-frequency noise introduced during analog-to-digital conversion (ADC), a simple averaging technique is applied at the microcontroller level. Multiple consecutive samples (e.g., 10 readings) are averaged to produce a single stable value.

This method:

- Reduces random noise fluctuations
- Improves signal stability
- Minimizes quantization noise effects

(b) Moving Average Filter

A moving average filter is applied in software to smooth the EMG signal. It is defined as:

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n - k]$$

Where,

- $x[n] \rightarrow$ input signal (EMG data)

- $y[n] \rightarrow$ filtered (smoothed) output
- $N \rightarrow$ window size (number of samples averaged)

This filter acts as a **low-pass filter**, reducing high-frequency variations while preserving the overall signal trend. It is particularly effective for EMG signals, where rapid fluctuations may not correspond to meaningful muscle activity.

(c) Signal Segmentation (Windowing)

Since EMG signals vary over time, the continuous data stream is divided into overlapping segments using a sliding window approach:

- Window size = 50 samples
- Step size = 25 samples (50% overlap)

This approach ensures:

- Temporal continuity in signal analysis
- Better capture of dynamic muscle activity
- Increased number of training samples

Each window is treated as an independent sample for feature extraction and classification.

3.5.2 Feature Extraction

Feature extraction transforms raw EMG signals into a compact and informative representation that captures essential characteristics of muscle activity. In this system, time-domain features are used due to their low computational complexity and effectiveness in real-time applications.

For each segmented window and for each EMG channel, the following features are computed:

1. Root Mean Square (RMS)

RMS reflects the signal energy and is directly related to muscle contraction force.

$$RMS = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$$

2. Mean Absolute Value (MAV)

MAV provides a measure of average signal amplitude and is useful for detecting muscle activation levels.

$$MAV = \frac{1}{N} \sum_{i=1}^N |x_i|$$

3. Waveform Length (WL)

WL represents the cumulative length of the waveform and captures both amplitude and frequency information.

$$WL = \sum_{i=1}^{N-1} |x_{i+1} - x_i|$$

4. Zero Crossing (ZC)

ZC provides an indication of signal frequency content and muscle activity transitions. It counts the number of times the signal crosses zero:

$$ZC = \sum_{i=1}^{N-1} (x_i \cdot x_{i+1} < 0)$$

5. Slope Sign Change (SSC)

SSC reflects signal complexity and frequency variations. It counts the number of times the slope of the signal changes direction:

$$SSC = \sum_{i=2}^{N-1} ((x_i - x_{i-1})(x_i - x_{i+1}) > 0)$$

6. Variance (VAR)

Variance measures signal power and dispersion around the mean.

$$VAR = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

Feature Vector Formation

Since five EMG sensors are used and six features are extracted from each sensor:

- Total features per window = $5 \times 6 = 30$ features

These features are arranged into a structured matrix (5×6), preserving spatial relationships across different muscle channels.

3.5.3 Feature Normalization

The extracted feature values may vary significantly in scale, which can negatively affect the performance of machine learning models. To address this, feature normalization is applied using a StandardScaler.

The transformation is given by:

$$X' = \frac{X - \mu}{\sigma}$$

where:

- μ is the mean
- σ is the standard deviation

Normalization ensures:

- All features contribute equally during training
- Faster convergence of the CNN model
- Improved numerical stability

The normalized feature matrix is then reshaped into a format suitable for CNN input ($5 \times 6 \times 1$), enabling spatial feature learning across channels.

In summary, the signal processing and feature extraction stage plays a critical role in transforming raw EMG signals into meaningful representations. By combining filtering, segmentation, and feature engineering, the system effectively captures the underlying muscle activity patterns required for accurate gesture classification.

3.6 Convolutional Neural Network (CNN) Architecture

A Convolutional Neural Network (CNN) is employed in this project to perform multi-class classification of EMG signal features into predefined hand gestures. CNNs are particularly effective in capturing spatial relationships and local feature patterns, making them suitable for structured EMG feature inputs.

In this system, the extracted feature vector (30 features) is reshaped into a 2D matrix of size $5 \times 6 \times 1$, where:

- 5 represents EMG channels (sensors)
- 6 represents extracted features per channel
- 1 represents a single feature map (grayscale-like input)

3.6.1 Neural Architecture

This structured representation enables the CNN to learn inter-channel and inter-feature relationships effectively.

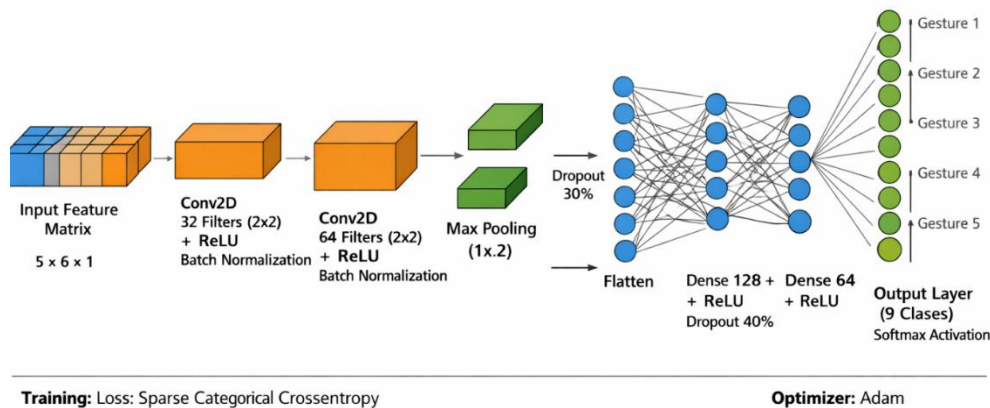


Fig. 3.11 Convolutional Neural Network (CNN) architecture for EMG-based gesture classification

1. Input Layer

The input to the CNN is a 3D tensor of shape: (5, 6, 1)

This format allows the network to treat the EMG feature matrix as a spatial input similar to an image.

2. First Convolutional Layer

- **Filters:** 32
- **Kernel Size:** (2×2)
- **Activation:** ReLU

This layer performs convolution to extract low-level spatial features from the EMG matrix. The small kernel size enables the network to capture local relationships between adjacent sensors and features.

ReLU Activation: $f(x) = \max(0, x)$

ReLU introduces non-linearity and helps the network learn complex patterns.

3. Batch Normalization

Batch normalization is applied after convolution to:

- Normalize feature distributions
- Accelerate training convergence
- Improve numerical stability

It reduces internal covariate shift and allows higher learning rates.

4. Second Convolutional Layer

- **Filters:** 64
- **Kernel Size:** (2×2)
- **Activation:** ReLU

This layer extracts higher-level features by combining patterns learned in the previous layer. Increasing the number of filters enhances the network's ability to learn complex representations.

5. Second Batch Normalization

Further stabilizes training and ensures consistent feature scaling across layers.

6. Max Pooling Layer (Pool Size: 1×2)

This layer reduces the spatial dimension of the feature maps, specifically along the feature axis, while preserving important information.

Benefits:

- Reduces computational complexity
- Prevents overfitting and retains dominant features

7. Dropout Layer (Dropout Rate: 0.3)

Dropout randomly disables a fraction of neurons during training, which helps:

- Prevent overfitting
- Improve generalization
- Reduce model dependency on specific neurons

8. Flatten Layer

The 2D feature maps are flattened into a 1D vector to prepare for fully connected layers:

feature maps → vector

9. Fully Connected (Dense) Layers

Dense Layer 1: Learns complex combinations of extracted features.

- Units: 128
- Activation: ReLU

Dropout Layer: Further reduces overfitting.

- Rate: 0.4

Dense Layer 2: Refines feature representation before classification.

- Units: 64
- Activation: ReLU

10. Output Layer

- **Units:** 9 (corresponding to 9 gestures)
- **Activation:** Softmax

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^9 e^{z_j}}$$

This layer outputs the probability distribution over all gesture classes.

3.6.2 Loss Function and Optimization

- **Loss Function:** Sparse Categorical Crossentropy

$$L = -\log(P(y_{\text{true}}))$$

- **Optimizer:** Adam (learning rate = 0.001)

Adam combines adaptive learning rates with momentum for efficient training.

3.6.3 Training Strategy

- **Train-Test Split:** 80% training, 20% testing
- **Validation Split:** 20% of training data
- **Batch Size:** 32
- **Epochs:** Up to 100

3.6.4 Model Performance

The trained CNN model achieves an overall classification accuracy of approximately **80%** on the test dataset. The performance is further evaluated using confusion matrix and ROC analysis to assess class-wise prediction capability.

3.6.5 Advantages of CNN for EMG Classification

- Captures spatial relationships between sensors and features
- Handles multi-dimensional input effectively
- Learns hierarchical feature representations
- Provides better generalization compared to traditional classifiers

3.7 Live Prediction and Finger Movement Control

The live prediction and finger movement control module is responsible for real-time interpretation of EMG signals and corresponding actuation of the prosthetic hand. This stage integrates the trained Convolutional Neural Network (CNN) model with the embedded control system to enable continuous and responsive operation.

3.7.1 Real-Time EMG Data Acquisition

During live operation, EMG signals are continuously acquired from five sensors and transmitted to the processing unit via serial communication. The incoming data stream consists of synchronized readings from all channels corresponding to different muscle activities.

The signals are stored in buffers and processed using a sliding window approach:

- Window size = 50 samples
- Step size = 25 samples

This ensures continuous signal analysis without interruption.

3.7.2 Real-Time Feature Extraction

For each window of EMG data:

- The signals are first smoothed using a moving average filter
- Time-domain features (RMS, MAV, WL, ZC, SSC, VAR) are extracted for each channel
- A total of 30 features (5 channels $\times$ 6 features) are generated

The feature vector is then normalized using a pre-trained StandardScaler to maintain consistency with the training phase.

3.7.3 Gesture Prediction Using CNN

The normalized feature vector is reshaped into a matrix of size $(5 \times 6 \times 1)$ and passed to the trained CNN model.

The CNN outputs a probability distribution across nine gesture classes. The predicted gesture is determined using:

$$\text{Gesture} = \arg \max(P_i)$$

where (P_i) is the probability of class (i).

Confidence-Based Filtering

To improve prediction reliability:

- A confidence threshold (e.g., 0.7) is applied
- Predictions below the threshold are ignored
- Repeated predictions are suppressed unless a change is detected

This reduces false triggering and ensures stable control.

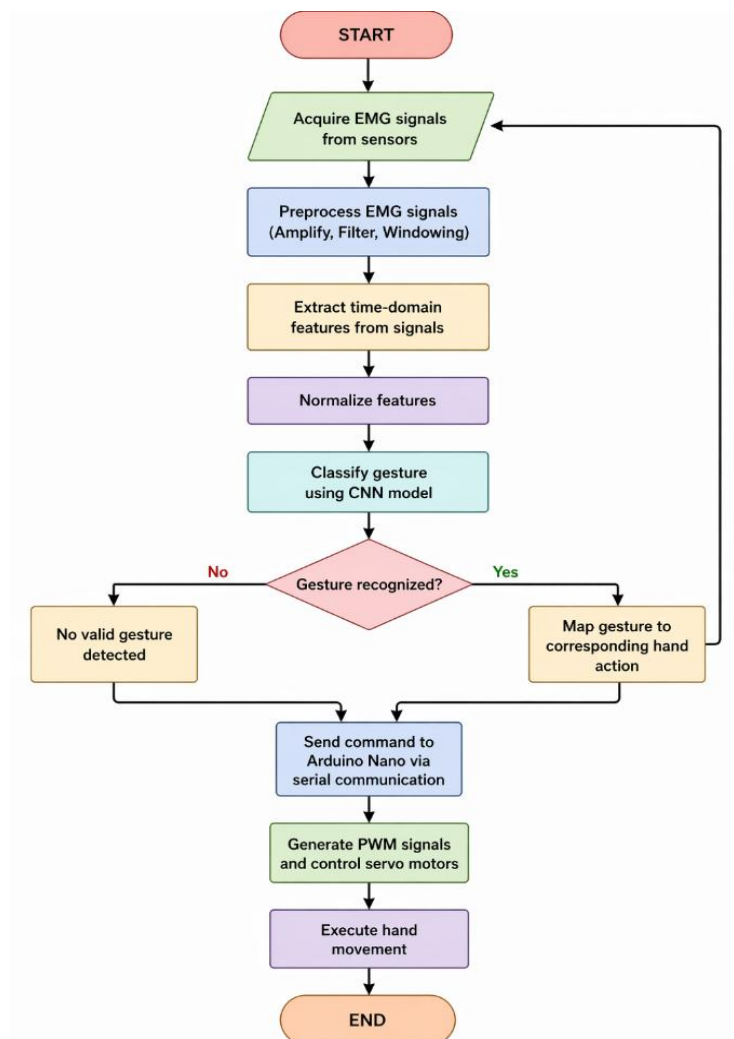


Fig. 3.12 Flowchart of the EMG-based prosthetic hand control system

3.7.4 Serial Communication to Arduino Nano

Once a valid gesture is detected:

- The gesture ID (0–8) is transmitted via serial communication
- Data is sent only when the predicted gesture changes
- This minimizes redundant commands and improves system efficiency

The Arduino Nano continuously listens for incoming serial data and decodes the gesture command.

3.7.5 Servo Motor Control Mechanism

The Arduino Nano controls five MG996R servo motors using PWM signals. Each motor is assigned to a specific finger:

1. Thumb
2. Index finger
3. Middle finger
4. Ring finger
5. Little finger

The servo position is controlled by varying the pulse width:

- $\sim 1 \text{ ms} \rightarrow 0^\circ$ (open position)
- $\sim 2 \text{ ms} \rightarrow 180^\circ$ (closed position)

3.7.6 Gesture-to-Movement Mapping

Each predicted gesture corresponds to a predefined motor configuration:

- **Open Hand:** All servos at open position
- **Close Hand:** All servos at closed position
- **Individual Finger Movement:** Only selected servo activated
- **Victory Sign:** Index and middle fingers open; others closed
- **Thumbs Up:** Thumb open; remaining fingers closed

This mapping enables both independent and combined finger movements.

3.7.7 Tendon-Driven Actuation

The servo motors actuate the fingers through a tendon-driven mechanism using nylon fishing lines. When a servo rotates:

- The tendon is pulled $\rightarrow$ finger flexes (closes)
- Releasing tension $\rightarrow$ finger extends (opens)

This mechanism mimics natural human hand motion and provides smooth and controlled movement.

3.7.8 Real-Time System Performance

The system operates in real-time with the following characteristics:

- Continuous EMG signal acquisition and processing
- Low-latency gesture prediction using sliding window technique
- Stable operation through confidence-based filtering
- Efficient communication between processing unit and Arduino

The integration of machine learning with embedded control enables responsive and intuitive prosthetic hand operation. This live prediction and control module forms the final stage of the system, converting classified EMG signals into physical movements.

3.8 Data Collection for Generic EMG Database

To improve the generalization capability of the proposed system, a structured approach for creating a generic EMG dataset is considered. Instead of relying solely on data from a single user, EMG signals can be collected from multiple individuals with varying physical characteristics such as age, muscle mass, skin impedance, and forearm anatomy.

During data collection, each participant performs a predefined set of gestures under controlled conditions. EMG signals from all sensors are recorded and labeled accordingly. By including data from diverse users, the dataset captures inter-subject variability, which is essential for developing a robust and user-independent model.

Such a generalized dataset is particularly important for practical prosthetic applications, where the end user may not be able to provide sufficient training data (e.g., individuals with limb loss). In these cases, a pre-trained model based on multi-user data can be deployed, allowing immediate usability with minimal calibration.

To further enhance adaptability, normalization techniques and feature scaling are applied to reduce subject-specific variations. This enables the trained model to learn generalized patterns of muscle activity rather than user-specific characteristics.

3.9 Summary

This chapter presented the complete methodology for the development of the EMG-based prosthetic hand system. The system integrates biomedical signal acquisition, analog and digital signal processing, machine learning, and embedded control to achieve real-time gesture recognition and actuation.

EMG signals are acquired from forearm muscles using multiple sensors, conditioned through amplification and filtering, and then digitized using a microcontroller. The signals undergo

preprocessing, segmentation, and feature extraction to generate meaningful representations of muscle activity. These features are normalized and classified using a Convolutional Neural Network (CNN) to identify different hand gestures.

The predicted gesture is transmitted to the Arduino Nano, which controls servo motors to replicate corresponding finger movements through a tendon-driven mechanism. The system is designed for real-time operation using sliding window processing and confidence-based filtering to ensure stable and accurate performance.

Additionally, the methodology incorporates both user-specific and generalized data approaches to improve adaptability and usability across different individuals. Overall, the proposed framework demonstrates an effective integration of hardware and software for intelligent prosthetic hand control.

Chapter 4: Implementation of a prosthetic arm

This section presents the performance evaluation of the proposed EMG-based prosthetic hand system. The results are analyzed for both the generic dataset model and the custom-trained model, followed by overall system performance evaluation.

4.1 EMG Signal Observation

In order to understand the behavior of EMG signals corresponding to different finger movements, the raw analog values obtained from the Arduino Nano were analyzed using the serial plotter. The EMG signals, after analog conditioning, were converted into digital values using a 10-bit Analog-to-Digital Converter (ADC), resulting in a range of 0 to 1023. The observed baseline value for all sensors was found to be approximately in the range of **500–530**, corresponding to the mid-supply voltage (~ 2.5 V), which represents the resting state of the muscles when no significant contraction is present.

Table 4.1 presents the observed ADC ranges for different finger movements across five EMG channels (thumb, index, middle, ring, and little). The following key observations were made:

- **Baseline Condition:** All sensors remain around 500–530 during rest, indicating stable signal acquisition.
- **High Muscle Activation:** Gestures such as *all close* produce significantly higher ADC values (up to ~ 900), reflecting strong muscle contraction.
- **Moderate Activation in Individual Movements:** Single finger movements show moderate increases in ADC values compared to baseline.
- **Overlapping Signal Ranges:** Gestures involving **index, middle, and ring fingers** exhibit overlapping ADC values due to anatomical proximity and muscle cross-talk, making them harder to distinguish using simple thresholding.
- **Variation Across Fingers:** Differences in ADC amplitudes are observed across fingers; the **middle finger** often produces higher values due to stronger muscle activation, while the **little finger** shows comparatively lower signal variation.
- **Effect of Sensor Placement:** Variability in signal amplitude is influenced by electrode placement and individual muscle characteristics.
- **Need for Advanced Classification:** Due to overlapping and non-linear signal patterns, threshold-based methods are insufficient, thereby necessitating feature extraction and CNN-based classification.

Overall, the ADC analysis confirms that although EMG signals exhibit distinguishable patterns for different gestures, their variability and overlap require intelligent signal processing and machine learning techniques to achieve accurate and reliable prosthetic control.

Table 4.1: Observed ADC value ranges for different finger movements across five EMG channels (thumb, index, middle, ring, and little)

Finger Movements	Thumb	Index	Middle	Ring	Little
All Open	500-530	500-530	500-530	500-530	500-530
All Close	800-900	680-800	600-780	600-750	550-650
Thumb	680-800	510-580	510-580	510-580	510-580
Index	510-580	750-915	580-600	580-600	510-580
Middle	510-520	580-600	800-1020	580-600	510-580
Ring	500-520	550-580	580-630	600-750	580-630
Little	500-520	520-560	530-580	550-600	600-700
Victory	680-800	550-600	550-600	630-750	600-700
Thumbs up	550-600	680-800	60-880	650-720	600-700

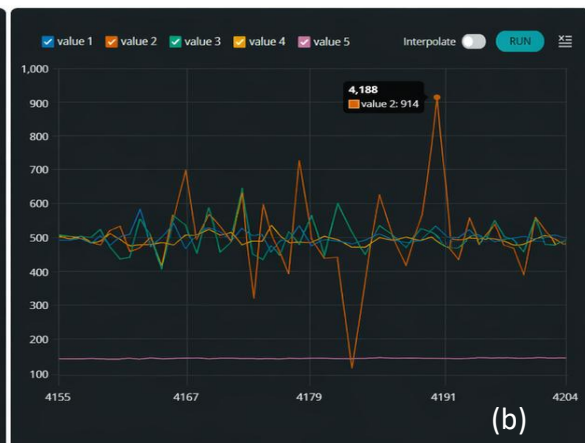
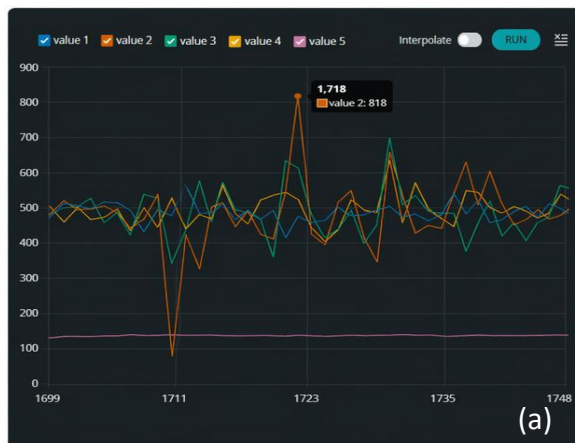




Figure 4.1 EMG signal patterns (ADC values) for different finger movements across five channels. (a) Ring Finger (b) Thumb (c) Little Finger (d) All Fingers Closed (e) Victory sign (f) Index Finger (g) Middle Finger (h) All Fingers Open

4.2 Results and Observations

4.2.1 Generic Model

Table 4.2: Generic Model Performance

Observations	Individual Movement Accuracy									Overall Accuracy
	Open	Close	Thumb	Index	Middle	Ring	Little	Victory	Thumbs Up	
Person 1	H	H	H	M	M	M	L	H	M	~80%
Person 2	H	H	H	M	M	L	L	M	L	~78%
Person 3	H	H	H	H	M	M	L	M	M	~82%
Person 4	H	H	H	M	M	M	M	H	M	~83%
Person 5	H	H	M	H	M	L	L	M	L	~76%
Person 6	H	M	M	M	M	M	M	H	L	~75%
Person 7	H	H	H	M	H	M	M	H	L	~85%
Person 8	H	H	H	M	M	M	L	M	M	~79%
Person 9	H	H	H	H	M	L	L	M	L	~77%
Person 10	H	H	H	M	H	M	M	H	L	~85%

H → High Accuracy M → Medium Accuracy L → Low Accuracy

Output Analysis for Generic Model

The generic dataset model achieved an overall accuracy of approximately **80%**, demonstrating its capability to generalize across multiple users with different physiological characteristics.

It is observed that gestures such as **open and close** consistently show high accuracy across all users. This is due to the strong and distinct muscle activation patterns associated with these movements, making them easier to classify.

In contrast, gestures involving **individual fingers (index, middle, ring, and little)** exhibit medium to low accuracy. This is primarily due to overlapping EMG signals and cross-talk between adjacent muscles, which makes it difficult to distinguish between similar finger movements.

Additionally, variation in accuracy across different individuals highlights the effect of **inter-subject variability**, including differences in muscle strength, electrode placement, and skin impedance.



Figure 4.2 Data Collection and Testing of Model

Despite these challenges, the model successfully captures generalized EMG patterns, making it suitable for applications where **user-specific calibration is not possible**, such as pre-trained prosthetic systems for amputees.

Overall, the results demonstrate that while the generic model provides scalability and usability, its performance is slightly lower compared to a custom-trained model due to the absence of personalized training.

4.2.2 Custom Model

Table 4.3 Custom Model Performance

Gesture	Accuracy (%)	Observation
Open	High (~98%)	Easily distinguishable
Close	High (~98%)	Strong muscle activation
Thumb	High (~90%)	Easily distinguishable
Index	Medium (~85%)	Similar patterns
Middle	Medium (~85%)	Overlap with index
Ring	Medium (~80%)	Weak signal
Little	Lower (~60%)	Hard to detect
Victory	Medium (~60%)	Mixed signals
Thumbs Up	Lower (~50)	Overlap with close

Output Analysis for Custom Model

The custom-trained model achieved an improved accuracy of approximately **85%**, indicating better performance compared to the generic dataset model. This improvement is attributed to user-specific training, which allows the model to learn more precise EMG signal patterns.

The confusion matrix analysis shows that:

- **Open, Close, and Thumbs Up** gestures are classified with high accuracy.
- Misclassification occurs mainly between **Index–Middle–Ring** gestures due to similar muscle activation.
- **Little finger and ring finger** gestures show lower accuracy due to weaker and less distinct EMG signals.

The ROC curve analysis demonstrates strong classification performance, with most classes achieving **AUC values close to 1.0**, indicating excellent separability between gesture classes. Slightly lower AUC values are observed for complex gestures, reflecting minor classification challenges.

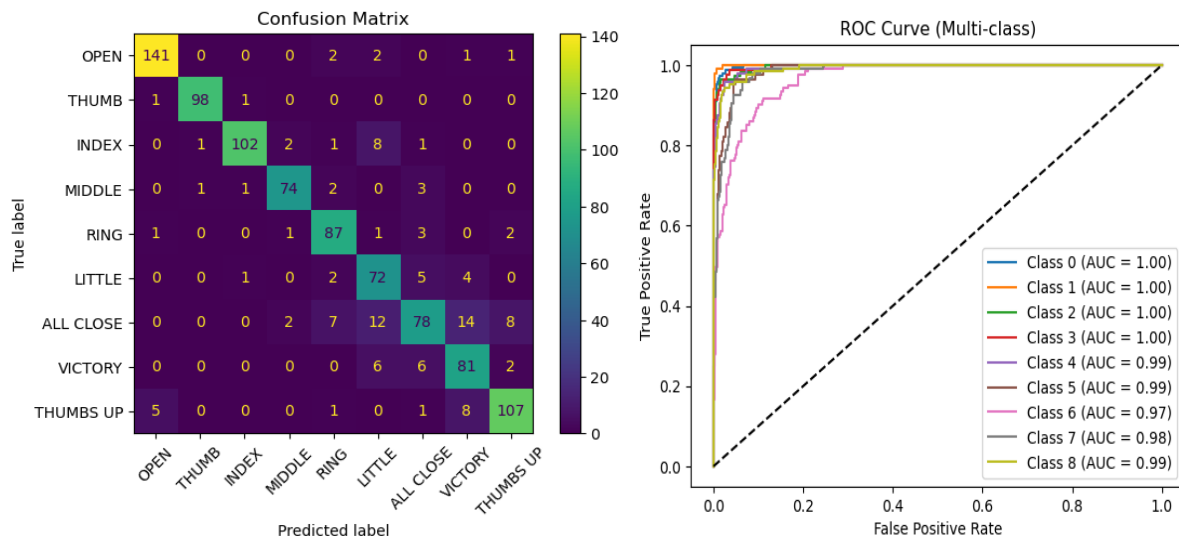


Figure 4.3 Confusion Matrix and ROC Curve

Overall, the custom model provides improved accuracy, stability, and better real-time performance compared to the generic dataset model.

4.3 System Performance

Table 4.4 Performance Metrics Comparison

Metric	Generic Model	Custom Model
Accuracy	80.112	85.443
Precision	78.724	87.474
Recall	77.121	87.331
F1-Score	76.462	81.764
Training Accuracy	82.246	86.664
Validation Accuracy	79.245	86.147
Loss	0.4442	0.3390

System Analysis

The system demonstrates effective real-time EMG signal classification using a CNN-based approach. The integration of signal preprocessing, feature extraction, and deep learning enables accurate gesture recognition.

Key observations include:

- The **custom model outperforms the generic model**, highlighting the importance of user-specific calibration.

- **Distinct gestures** (open/close) achieve higher accuracy due to strong muscle activation.
- **Similar finger movements** lead to misclassification due to overlapping EMG patterns.
- The **multi-channel EMG setup (5 sensors)** improves classification capability compared to single-channel systems.
- The **sliding window technique and confidence threshold (0.7)** ensure stable real-time predictions.
- The system achieves **low latency and smooth servo response**, enabling practical prosthetic hand control.

4.4 Summary

The performance evaluation of the proposed EMG-based prosthetic hand system demonstrates the effectiveness of integrating multi-channel EMG acquisition, signal processing, and CNN-based classification for real-time gesture recognition. The raw EMG signals, after analog conditioning and ADC conversion, exhibited a stable baseline around 500–530, with distinct variations corresponding to different finger movements. However, overlapping signal ranges among adjacent fingers highlighted the limitations of threshold-based approaches and justified the use of machine learning. The generic model achieved an overall accuracy of approximately **80%**, demonstrating good generalization across multiple users despite inter-subject variability. In contrast, the custom-trained model achieved improved performance of approximately **85%**, with higher precision, recall, and F1-score, indicating more accurate and consistent classification due to user-specific training.

Analysis of the confusion matrix and ROC curves confirmed strong classification capability, with most gesture classes showing high separability ($AUC \approx 1.0$). Misclassification was primarily observed among gestures involving similar muscle groups, such as index, middle, and ring fingers, due to signal overlap and cross-talk.

Overall system performance metrics indicate balanced and reliable operation, with low loss values and minimal latency. The multi-channel EMG setup, combined with sliding window processing and confidence-based filtering, enabled stable real-time control of servo-driven finger movements.

In summary, the proposed system successfully achieves a trade-off between accuracy, cost, and real-time performance, demonstrating its potential as an efficient and scalable solution for intelligent prosthetic hand applications.

Chapter 5: Conclusions and Future Works

5.1 Conclusion

This project presented the design and implementation of an EMG-based prosthetic hand system using a Convolutional Neural Network (CNN) for gesture classification. The system integrates biomedical signal acquisition, analog and digital signal processing, machine learning, and embedded control to achieve real-time prosthetic hand actuation.

Surface EMG signals were acquired using five Muscles BioAmp Patchy sensors placed on the forearm. The signals were conditioned using differential amplification and bandpass filtering, followed by digital preprocessing and segmentation. Time-domain features—RMS, MAV, WL, ZC, SSC, and VAR—were extracted and structured into a 5×6 feature matrix for classification. A CNN model was developed to classify nine distinct hand gestures, including individual finger movements and combined gestures. The model achieved an accuracy of approximately **80%** using a generic dataset and up to **~85–87%** with a custom-trained dataset. The corresponding precision, recall, and F1-score values indicate balanced and reliable classification performance across gesture classes.

The classified output was transmitted to an Arduino Nano, which controlled five MG996R servo motors to replicate finger movements through a tendon-driven mechanism. The system demonstrated stable real-time performance using a sliding window approach and confidence-based filtering, ensuring smooth and responsive prosthetic control.

The results highlight that gestures with strong and distinct muscle activation (e.g., open and close) achieved higher accuracy, while gestures involving adjacent fingers showed occasional misclassification due to overlapping EMG signals. Despite these challenges, the system successfully demonstrates the feasibility of combining EMG-based human-machine interfaces with deep learning for prosthetic applications. Overall, the proposed system provides a **cost-effective, intelligent, and scalable solution** for prosthetic hand control.

5.2 Future Work

While the proposed system demonstrates effective EMG-based prosthetic control, several enhancements can be implemented to further improve performance, usability, and real-world applicability.

1. Improved Accuracy Using Advanced Models

Future work can explore more advanced deep learning architectures such as hybrid

CNN-LSTM models or transformer-based approaches to better capture temporal dependencies in EMG signals and improve classification accuracy.

2. Expansion of Gesture Set

The current system supports nine gestures. This can be extended to include more complex and functional hand movements, enabling finer control and improved usability in daily activities.

3. Adaptive and Personalized Learning

Incorporating adaptive learning techniques or transfer learning can allow the system to automatically adjust to new users, reducing the need for manual calibration and improving user experience.

4. Development of a Large-Scale Generic Dataset

Creating a comprehensive multi-user EMG dataset will enhance model generalization and enable deployment for amputees without requiring extensive individual training.

5. Integration with IoT and Smart Systems

The system can be integrated with IoT platforms for remote monitoring, performance analysis, and cloud-based model updates, enabling continuous improvement and data-driven optimization.

6. Miniaturization and Embedded AI Implementation

Future versions can implement the model directly on embedded platforms (e.g., edge AI devices) to eliminate the need for an external computer and achieve fully portable operation.

7. Enhanced Sensor Technology

Using higher-quality EMG sensors or increasing the number of channels can improve signal resolution and reduce misclassification, especially for similar finger movements.

8. Mechanical Design Improvements

Refinement of the tendon-driven mechanism and use of lightweight materials can enhance durability, response time, and overall user comfort.

9. Clinical Testing and User Trials

Extensive testing with amputee users in real-world scenarios will help validate system performance, identify practical challenges, and improve design usability.

10. Noise Reduction and Signal Enhancement Techniques

Advanced filtering and signal processing techniques can be implemented to further reduce noise and improve robustness in dynamic environments.

References

- [1] A. Korai, A. A. Jamali, and A. Ahmed, “Using 3D Printing Technology to Develop Electromyography Signals Based Prosthetic Arm,” *Materials Today: Proceedings*, vol. 58, pp. 510–515, 2022.
- [2] M. Ashik Ali *et al.*, “Design and Development of EMG Based Prosthetic Arm,” *International Journal of Engineering Research & Technology (IJERT)*, 2020.
- [3] A. Unanyan and A. Belov, “Low-Price Prosthetic Hand Controlled by EMG Signals,” *IEEE Conference*, 2021.
- [4] M. R. Khan *et al.*, “Supervised Machine Learning Based Fast Hand Gesture Recognition Using EMG Signals,” *IEEE Access / Conference Proceedings*, 2021.
- [5] Y. Xu, X. Yin, Y. Fan, and Y. Qian, “Tendon-drive finger mechanisms for an EMG prosthetic hand with two motors,” *Mechanism and Machine Theory*, vol. 78, pp. 79–93, 2014.
- [6] L. E. Osborn *et al.*, “Extended home use of an advanced osseointegrated prosthetic arm improves function, performance, and control efficiency,” *Journal of Neural Engineering*, vol. 18, no. 2, p. 026020, 2021.
- [7] N. Sakib and M. K. Islam, “Design and Implementation of an EMG Controlled 3D Printed Prosthetic Arm,” in *IEEE BECITHCON*, 2019.
- [8] D. Karabulut, F. Ortes, Y. Z. Arslan, and M. A. Adli, “Comparative evaluation of EMG signal features for myoelectric controlled human arm prosthetics,” *Biocybernetics and Biomedical Engineering*, vol. 37, no. 2, pp. 326–335, 2017.
- [9] O. W. Samuel *et al.*, “Intelligent EMG Pattern Recognition Control Method for Upper-Limb Multifunctional Prostheses: Advances, Current Challenges, and Future Prospects,” *IEEE Access*, vol. 7, pp. 10150–10165, 2019.
- [10] L. S. Praveen, S. N. Nagananda, and P. Shankapal, “Design and Development of Real-Time Bionic Hand Control Using EMG Signal,” in *IEEE CONECCT*, 2018.
- [11] Upside Down Labs, “Muscle BioAmp Patchy,” GitHub Repository. Available: <https://github.com/upsidedownlabs/Muscle-BioAmp-Patchy>
- [12] Backyard Brains, “Experiment: Signal Classification (Robot Hand),” Documentation. Available: <https://docs.backyardbrains.com/retired/experiments/RobotHand/>
- [13] Arduino, “Arduino Nano Hardware Overview,” Official Documentation. Available: <https://docs.arduino.cc/hardware/nano/>
- [14] C. Melchiorri, G. Palli, G. Berselli, and G. Vassura, “Development of the UB Hand IV,” University of Bologna, 2013.
- [15] Handson Technology, “MG996R Metal Gear Servo Motor Datasheet,” Available: https://www.handsontec.com/dataspecs/motor_fan/MG996R.pdf.

- [16] C. Pylatiuk, S. Schulz, and L. Döderlein, "Results of an Internet survey of myoelectric prosthetic hand users," *Prosthetics and Orthotics International*, 2007.
- [17] C. Sapsanis, G. Georgoulas, and A. Tzes, "EMG-based classification of basic hand movements using time-frequency features," in *IEEE MED Conference*, 2013.
- [18] M. R. Ahsan, M. I. Ibrahimy, and O. O. Khalifa, "EMG signal-based hand gesture recognition using artificial neural network," in *IEEE ICOM*, 2011.
- [19] M. Z. Jamal, "Signal acquisition using surface EMG and circuit design considerations for robotic prosthesis," 2012.
- [20] P. Visconti *et al.*, "Technical features and functionalities of Myo armband," *International Journal of Smart Sensors*, 2018.
- [21] D. Yang *et al.*, "An anthropomorphic robot hand developed based on underactuated mechanism and controlled by EMG signals," *Journal of Bionic Engineering*, 2009.
- [22] F. Tenore *et al.*, "Control of individual fingers of a prosthetic hand using surface EMG signals," in *IEEE EMBS*, 2007.
- [23] R. N. Khushaba *et al.*, "Toward improved control of prosthetic fingers using EMG signals," *Expert Systems with Applications*, 2012.
- [24] M. Yoshikawa, M. Mikawa, and K. Tanaka, "Real-time hand motion estimation using EMG signals with SVM," in *IEEE Conference*, 2006.
- [25] E. A. Chung and M. E. Benalcázar, "Real-time hand gesture recognition using deep learning and EMG signals," in *EUSIPCO*, 2019.
- [26] B. Crawford *et al.*, "Real-time classification of EMG signals for robotic control," *AAAI*, 2005.
- [27] V. Shanmuganathan *et al.*, "R-CNN and wavelet feature extraction for EMG gesture recognition," *Neural Computing and Applications*, 2020.
- [28] W. Caesarendra, T. Tjahjowidodo, and D. Pamungkas, "EMG-based classification using PCA and ANFIS," in *IEEE Robionetics*, 2017.
- [29] M. U. Khan, S. Z.-Z. Ali, A. Ishtiaq, K. Habib, T. Gul, and A. Samer, "Classification of Multi-Class Cardiovascular Disorders Using Ensemble Classifier and Impulsive Domain Analysis," in *MAJICC 2021*, pp. 1–8, 2021.
- [30] M. U. Khan, A. Farman, A. ur Rehman, N. Israr, M. Z. H. Ali, and Z. A. Gulshan, "Automated System Design for Classification of Chronic Lung Viruses Using Non-Linear Dynamic System Features and K-Nearest Neighbour," in *MAJICC 2021*, pp. 1–8, 2021.
- [31] M. U. Khan, D. S. A. A. Zaidi, A. Ishtiaq, S. U. R. Bukhari, S. Samer, and A. Farman, "A Comparative Survey of LiDAR-SLAM and LiDAR-Based Sensor Technologies," in *MAJICC 2021*, pp. 1–8, 2021.
- [32] M. U. Khan, A. Mobeen, S. Samer, and A. Samer, "Embedded System Design for Real-Time Detection of Asthmatic Diseases Using Lung Sounds in Cepstral Domain," in *IEEC 2021*, pp. 1–5, 2021.

- [33] A. Zainab, M. U. Khan, H. Khan, and S. U. R. Bukhari, "Intelligent System Design for Early Diagnosis of Faults in Machine Bearings," in *IEEC 2021*, pp. 1–5, 2021.
- [34] S. Z. H. Naqvi, M. Arooj, S. Aziz, M. U. Khan, M. A. Choudhary, and M. N. Ul Hassan, "Spectral Analysis of Lung Sounds for Classification of Asthma and Pneumonia Wheezing," in *ICECCE 2020*, 2020.
- [35] A. Ishtiaq, M. U. Khan, S. Z. Ali, K. Habib, S. Samer, and E. Hafeez, "A Review of System-on-Chip Applications in Internet of Things and Medical Systems," in *ICAME 2021*, pp. 1–10, 2021.
- [36] M. U. Khan, S. Z. Ali, K. Habib, H. Khan, F. Tariq, and S. Bibi, "A Novel Intelligent Model for COVID-19 Detection Using Cough Auscultations and Hjorth Descriptors," in *ICAME 2021*, pp. 1–10, 2021.
- [37] P. Jonna, A. Madurwar, A. B. K., and M. Rao, "Design of 6-DOF Holonomic Drive-Based Upper and Lower Limb Stroke Rehabilitation System," in *ICARA 2024*, (in press).
- [38] V. L. Feigin, B. Norrving, and G. A. Mensah, "Global Burden of Stroke," *Circulation Research*, vol. 120, no. 3, pp. 439–448, 2017.
- [39] G. A. Roth *et al.*, "Global, Regional, and National Burden of Cardiovascular Diseases," *Journal of the American College of Cardiology*, vol. 70, no. 1, pp. 1–25, 2017.
- [40] C. L. Richards, F. Malouin, and S. Nadeau, "Stroke Rehabilitation: Clinical Picture, Assessment, and Therapeutic Challenge," *Progress in Brain Research*, vol. 218, pp. 253–280, 2015.
- [41] M. Lotze *et al.*, "Motor Learning Elicited by Voluntary Drive," *Brain*, vol. 126, no. 4, pp. 866–872, 2003.
- [42] World Health Organization, "Noncommunicable Diseases," WHO Report, 2011.
- [43] J. Bernhardt *et al.*, "Stroke Rehabilitation in Low- and Middle-Income Countries," *The Lancet*, vol. 396, pp. 1452–1462, 2020.
- [44] F. Gimigliano and S. Negrini, "Rehabilitation 2030: A Call for Action," *European Journal of Physical and Rehabilitation Medicine*, vol. 53, no. 2, pp. 155–168, 2017.
- [45] J. D. Pandian *et al.*, "Stroke Systems of Care in Low-Income Countries," *The Lancet*, vol. 396, pp. 1443–1451, 2020.
- [46] P. S. Lum *et al.*, "Robot-Assisted Movement Training vs Conventional Therapy," *Archives of Physical Medicine and Rehabilitation*, vol. 83, no. 7, pp. 952–959, 2002.
- [47] B. Husemann *et al.*, "Effects of Robot-Assisted Gait Training in Stroke Patients," *Stroke*, vol. 38, no. 2, pp. 349–354, 2007.
- [48] P. Jonna and M. Rao, "Design of a Cost-Effective Robotic System for Upper Limb Rehabilitation," in *IEEE EMBC*, pp. 1423–1427, 2022.
- [49] V. Chandrasekhar *et al.*, "Design of a Portable Anthropomorphic Upper Limb Rehabilitation Device," in *IEEE EMBC*, pp. 4708–4712, 2020.
- [50] K. Vinay *et al.*, "Design of a Device for Lower Limb Prophylaxis and Exercise," *IEEE Journal of Translational Engineering in Health and Medicine*, vol. 9, 2020.

- [51] L. Marchal-Crespo and D. J. Reinkensmeyer, "Review of Control Strategies for Robotic Movement Training," *Journal of NeuroEngineering and Rehabilitation*, vol. 6, 2009.
- [52] T. Proietti *et al.*, "Upper-Limb Robotic Exoskeletons for Neurorehabilitation," *IEEE Reviews in Biomedical Engineering*, vol. 9, pp. 4–14, 2016.
- [53] A. Frisoli *et al.*, "Development of a New Exoskeleton for Upper Limb Rehabilitation," in *IEEE Rehabilitation Robotics*, pp. 188–193, 2009.
- [54] P. Staubli *et al.*, "Effects of Intensive Arm Training Using ARMin II Robot," *Journal of NeuroEngineering and Rehabilitation*, 2009.
- [55] D. Lynch *et al.*, "Continuous Passive Motion Improves Shoulder Joint Integrity," *Clinical Rehabilitation*, vol. 19, no. 6, pp. 594–599, 2005.
- [58] N. Hogan *et al.*, "Motions or Muscles? Behavioral Factors in Robotic Assistance," *Journal of Rehabilitation Research and Development*, vol. 43, no. 5, 2006.
- [59] R. Colombo *et al.*, "Robotic Techniques for Upper Limb Rehabilitation," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 13, no. 3, pp. 311–324, 2005.
- [60] J. L. Patton and F. A. Mussa-Ivaldi, "Robot-Assisted Adaptive Training," *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 4, pp. 636–646, 2004.
- [61] Symbionic, "Advanced Myoelectric Prosthetic Hand," 2024. [Online]. Available: <https://www.symbionic.co/>
- [62] IndiaMART, "Robotic Prosthetic Arm," 2024. [Online]. Available: <https://www.indiamart.com/proddetail/robotic-prosthetic-arm-23578610997.html>
- [63] Swami Vivekanand National Institute of Rehabilitation Training and Research (SVNIRTAR), Olatpur, Government of India Prosthetic Services.

Appendix A: Source Code

Arduino Code for Data Collection

```
#define EMG_THUMB A0
#define EMG_INDEX A1
#define EMG_MIDDLE A2
#define EMG_RING A3
#define EMG_LITTLE A4
int readEMG(int pin) {
  int sum = 0;
  for(int i = 0; i < 10; i++) {
    sum += analogRead(pin);
  }
  return sum / 10;
}
void setup() {
  Serial.begin(115200);
}
void loop() {
  int t = readEMG(EMG_THUMB);
  int i = readEMG(EMG_INDEX);
  int m = readEMG(EMG_MIDDLE);
  int r = readEMG(EMG_RING);
  int l = readEMG(EMG_LITTLE);
  Serial.print(t); Serial.print(",");
  Serial.print(i); Serial.print(",");
  Serial.print(m); Serial.print(",");
  Serial.print(r); Serial.print(",");
  Serial.println(l);
  delay(5);
}
```

Python Code for Data Collection and Feature Extraction

```
import serial, csv, numpy as np, time, msvcrt
PORT = 'COM5'
ser = serial.Serial(PORT, 115200)
time.sleep(2)
WINDOW_SIZE = 50
STEP = 25
def extract_features(signal):
    signal = np.array(signal)
    return [
        np.sqrt(np.mean(signal**2)),
        np.mean(np.abs(signal)),
        np.sum(np.abs(np.diff(signal))),
        np.sum(signal[:-1] * signal[1:] < 0),
        np.sum(((signal[1:-1] > signal[:-2]) & (signal[1:-1] > signal[2:])) |
                ((signal[1:-1] < signal[:-2]) & (signal[1:-1] < signal[2:]))),
        np.var(signal)
    ]
def smooth(signal):
```



```

    return np.convolve(signal, np.ones(5)/5, mode='valid')
print("\nLabels:")
print("0 Open | 1 Thumb | 2 Index | 3 Middle | 4 Ring | 5 Little | 6 All Close | 7 Victory | 8
ThumbsUp")
current_label = None
with open("emg_dataset_features.csv", "w", newline="") as f:
    writer = csv.writer(f)
    header = []
    for s in ['thumb', 'index', 'middle', 'ring', 'little']:
        header += [f'{s}_rms', f'{s}_mav', f'{s}_wl', f'{s}_zc", f'{s}_ssc", f'{s}_var"]
    header.append("label")
    writer.writerow(header)
    bt, bi, bm, br, bl = [], [], [], [], []
    while True:
        if ser.in_waiting:
            try:
                line = ser.readline().decode().strip()
                parts = line.split(",")
                if len(parts) != 5:
                    continue
                t, i, m, r, l = map(int, parts)
                bt.append(t)
                bi.append(i)
                bm.append(m)
                br.append(r)
                bl.append(l)
                if len(bt) >= WINDOW_SIZE and current_label is not None:
                    features = []
                    features += extract_features(smooth(bt))
                    features += extract_features(smooth(bi))
                    features += extract_features(smooth(bm))
                    features += extract_features(smooth(br))
                    features += extract_features(smooth(bl))
                    features.append(current_label)
                    writer.writerow(features)
                    print(f"Saved label {current_label}")
                    bt = bt[STEP:]
                    bi = bi[STEP:]
                    bm = bm[STEP:]
                    br = br[STEP:]
                    bl = bl[STEP:]
            except:
                pass
    if msvcrt.kbhit():
        key = msvcrt.getch().decode()
        if key == 'q':
            break
        elif key in [str(i) for i in range(9)]:
            current_label = int(key)
            print(f"\nRecording {current_label}")

```

Python Code for Model Training and Model Performance

```
import pandas as pd
import numpy as np
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout,
BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.optimizers import Adam

data = pd.read_csv(r"C:\Users\arush\OneDrive\Desktop\EMG\5 Sensor\emg_dataset_features.csv")
X = data.iloc[:, :-1].values # features
y = data.iloc[:, -1].values # labels
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

joblib.dump(scaler, "scaler.pkl")
X_train = X_train.reshape(-1, 5, 6, 1)
X_test = X_test.reshape(-1, 5, 6, 1)
model = Sequential([
    Conv2D(32, (2,2), activation='relu', input_shape=(5,6,1)),
    BatchNormalization(),
    Conv2D(64, (2,2), activation='relu'),
    BatchNormalization(),
    MaxPooling2D((1,2)),
    Dropout(0.3),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.4),
    Dense(64, activation='relu'),
    Dense(9, activation='softmax') # 9 gestures
])
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)
lr_reduce = ReduceLROnPlateau(
    monitor='val_loss',
```

```

    factor=0.5,
    patience=5,
    min_lr=1e-5
)
history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop, lr_reduce],
    verbose=1
)
loss, accuracy = model.evaluate(X_test, y_test)
print("\nFinal Test Accuracy:", accuracy)
y_pred = np.argmax(model.predict(X_test), axis=1)
model.save("emg_cnn_model.h5")
print("Model and scaler saved successfully!")
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
classification_report
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall  :", recall)
print("F1 Score :", f1)

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
y_pred_probs = model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)
cm = confusion_matrix(y_test, y_pred)

labels = [ "OPEN", "THUMB", "INDEX", "MIDDLE", "RING", "LITTLE", "ALL CLOSE", "VICTORY", "THUMBS
UP"]
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(xticks_rotation=45)

plt.title("Confusion Matrix")
plt.show()
y_test_bin = label_binarize(y_test, classes=np.arange(9))
plt.figure()
for i in range(9):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_pred_probs[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f"Class {i} (AUC = {roc_auc:.2f})")
plt.plot([0,1], [0,1], 'k--')

```

```

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve (Multi-class)")
plt.legend()
plt.show()

```

Python Code for Live Classification for Movements

```

import serial
import numpy as np
import joblib
import time
from tensorflow.keras.models import load_model

PORT = 'COM5'
ser = serial.Serial(PORT, 115200)
time.sleep(2)
model = load_model("5 Sensor\emg_cnn_model.h5")
scaler = joblib.load("5 Sensor\scaler.pkl")
WINDOW_SIZE = 50
STEP = 25

def extract_features(signal):
    signal = np.array(signal)
    rms = np.sqrt(np.mean(signal**2))
    mav = np.mean(np.abs(signal))
    wl = np.sum(np.abs(np.diff(signal)))
    zc = np.sum(signal[:-1] * signal[1:] < 0)
    ssc = np.sum(
        ((signal[1:-1] > signal[:-2]) & (signal[1:-1] > signal[2:])) |
        ((signal[1:-1] < signal[:-2]) & (signal[1:-1] < signal[2:]))
    )
    var = np.var(signal)
    return [rms, mav, wl, zc, ssc, var]

def smooth(signal):
    return np.convolve(signal, np.ones(5)/5, mode='valid')

labels = [ "OPEN", "THUMB", "INDEX", "MIDDLE", "RING", "LITTLE", "ALL CLOSE", "VICTORY",
"THUMBS UP"]
bt, bi, bm, br, bl = [], [], [], [], []
last_sent = -1 # avoid repeating same command
print(" Live EMG Prediction Started...\n")
while True:
    if ser.in_waiting:
        try:
            line = ser.readline().decode().strip()
            parts = line.split(",")
            if len(parts) != 5:
                continue
            t, i, m, r, l = map(int, parts)
            bt.append(t)
            bi.append(i)

```

```

bm.append(m)
br.append(r)
bl.append(l)
if len(bt) >= WINDOW_SIZE:
    features = []
    features += extract_features(smooth(bt))
    features += extract_features(smooth(bi))
    features += extract_features(smooth(bm))
    features += extract_features(smooth(br))
    features += extract_features(smooth(bl))
    X = scaler.transform([features])
    X = X.reshape(-1, 5, 6, 1)
    pred = model.predict(X, verbose=0)
    gesture_id = int(np.argmax(pred))
    gesture = labels[gesture_id]
    confidence = np.max(pred)
    print(f"Prediction: {gesture} ({confidence:.2f})")
    if gesture_id != last_sent and confidence > 0.7:
        ser.write(f"{gesture_id}\n".encode())
        ser.flush()
        last_sent = gesture_id
    bt = bt[STEP:]
    bi = bi[STEP:]
    bm = bm[STEP:]
    br = br[STEP:]
    bl = bl[STEP:]
    time.sleep(0.05)
except Exception as e:
    print("Error:", e)

```

Arduino Code for Driving Motor

```

#include <Servo.h>
#define EMG_THUMB A0
#define EMG_INDEX A1
#define EMG_MIDDLE A2
#define EMG_RING A3
#define EMG_LITTLE A4
Servo thumb, indexF, middle, ring, little;
int posOpen = 0;
int posClose = 170;
int readEMG(int pin) {
    int sum = 0;
    for (int i = 0; i < 10; i++) {
        sum += analogRead(pin);
    }
    return sum / 10;
}
void setup() {
    Serial.begin(115200);
    thumb.attach(3);
    indexF.attach(5);
    middle.attach(6);

```

```

    ring.attach(9);
    little.attach(10);
    openHand();
}
void loop() {
    int t = readEMG(EMG_THUMB);
    int i = readEMG(EMG_INDEX);
    int m = readEMG(EMG_MIDDLE);
    int r = readEMG(EMG_RING);
    int l = readEMG(EMG_LITTLE);
    Serial.print(t); Serial.print(",");
    Serial.print(i); Serial.print(",");
    Serial.print(m); Serial.print(",");
    Serial.print(r); Serial.print(",");
    Serial.println(l);
    if (Serial.available()) {
        int gesture = Serial.parseInt();
        executeGesture(gesture);
    }
    delay(5);
}

```

```

void executeGesture(int gesture) {
    switch (gesture) {
        case 0: openHand(); break;
        case 1: thumbOnly(); break;
        case 2: indexOnly(); break;
        case 3: middleOnly(); break;
        case 4: ringOnly(); break;
        case 5: littleOnly(); break;
        case 6: closeHand(); break;
        case 7: victory(); break;
        case 8: thumbsup(); break;
    }
}

void openHand() {
    thumb.write(posOpen);
    indexF.write(posOpen);
    middle.write(posOpen);
    ring.write(posOpen);
    little.write(posOpen);
}

void closeHand() {
    thumb.write(posClose);
    indexF.write(posClose);
    middle.write(posClose);
    ring.write(posClose);
    little.write(posClose);
}

void thumbOnly() {
    openHand();
}

```

```

    thumb.write(posClose);
}

void indexOnly() {
    openHand();
    indexF.write(posClose);
}

void middleOnly() {
    openHand();
    middle.write(posClose);
}

void ringOnly() {
    openHand();
    ring.write(posClose);
}

void littleOnly() {
    openHand();
    little.write(posClose);
}

void victory() {
    openHand();
    thumb.write(posClose);
    ring.write(posClose);
    little.write(posClose);
}

void thumbsup() {
    openHand();
    indexF.write(posClose);
    middle.write(posClose);
    ring.write(posClose);
    little.write(posClose);
}

```